



CSE 4205

Digital Logic Design

Combinational Logic with MSI & LSI

Course Teacher: Md. Hamjajul Ashmafee

Lecturer, CSE, IUT

Email: ashmafee@iut-dhaka.edu



Remaining to Add

- Review other books and online resources



Review

We have already learned about

- **Binary numbers** and **Binary codes** are representation of discrete quantities of information.
- These **binary variables (signals)** are applied by electric voltages or some other means.
- This signal can be manipulated through different **digital logic gates** to perform required functions.
- **Boolean algebra** is used to express and manipulate the Boolean functions
- Several **methods** (K-map, tabular method,...) can be used **to simplify** the Boolean functions and optimize the gate implementations

*In this chapter, we will learn how to formulate various **systematic design** and analysis the **procedures** of combinational circuits.*

Introduction

- Logic circuits of any digital system may be any of:
 - Combinational Logic Circuit
 - Sequential Logic Circuit
- A **combinational circuit** is the circuit whose outputs at any time are **determined** directly from the **present combination of inputs** without regarding to previous inputs or outputs.
 - Operation of any combinational circuit can be specified by a set of Boolean functions
- On the other hand, **Sequential circuits** employ **memory elements** (binary memory cells) in addition to the logic gates.
 - Their **outputs** are the **function** of the **present inputs** and the **state of the memory elements**.
 - Its behavior must be specified by a **time sequence** of inputs and internal states of memory.

Combinational Circuit

- Combinational circuits consists of interconnection of logic gates
 - Logic gates react to the values of the signal at the inputs and produce required output based on the definition
 - These input and output signal exists physically as analog signal which must be interpreted as a binary signal representing either logic-1 or logic-0
- But, In many application, the source and destination of the combination circuit are storage registers
 - If the registers are included with the combinational circuit, then total circuit must be considered as a *sequential circuit*
- **Note:** A sequential circuit is circuit which has logic gates with feedback path or memory elements
 - A feedback path is a connection from output of any logic gate partially forms an input to any previous gate

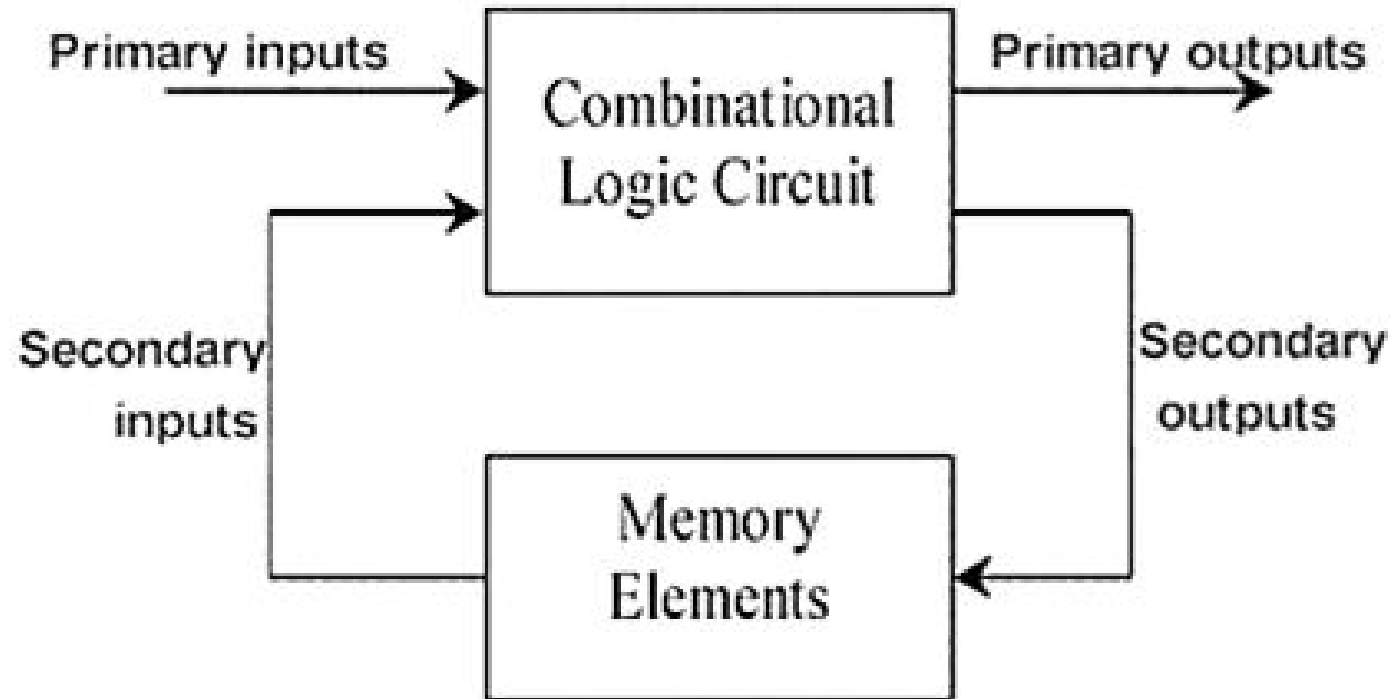
Combinational Circuit: Block Diagram

- Block diagram of a combinational circuit with n input and m output binary variables:



- For n input variables, there are 2^n possible combinations of the binary inputs (from external sources)
 - For each input combination, there is one possible value for each output variable (to external destinations)
- A combinational circuit can be specified as a truth table that enlists output values for each input combination
 - It is also specified as a Boolean function expressed in terms of the n input variables

Combinational and Sequential Circuit



Analysis of a Combinational Circuit

- Analysis of any combinational circuit starts with a logic diagram
 - Finally we will **determine the function** that the circuit implements, **generates the truth table** and feasibly **explains the operation** of the circuit.
 - If the function name or its explanation are already mentioned with logic circuit, then we need to verify it (**verification**)
 - Another way to analyze any circuit with **logic simulation** if the number of inputs is very large

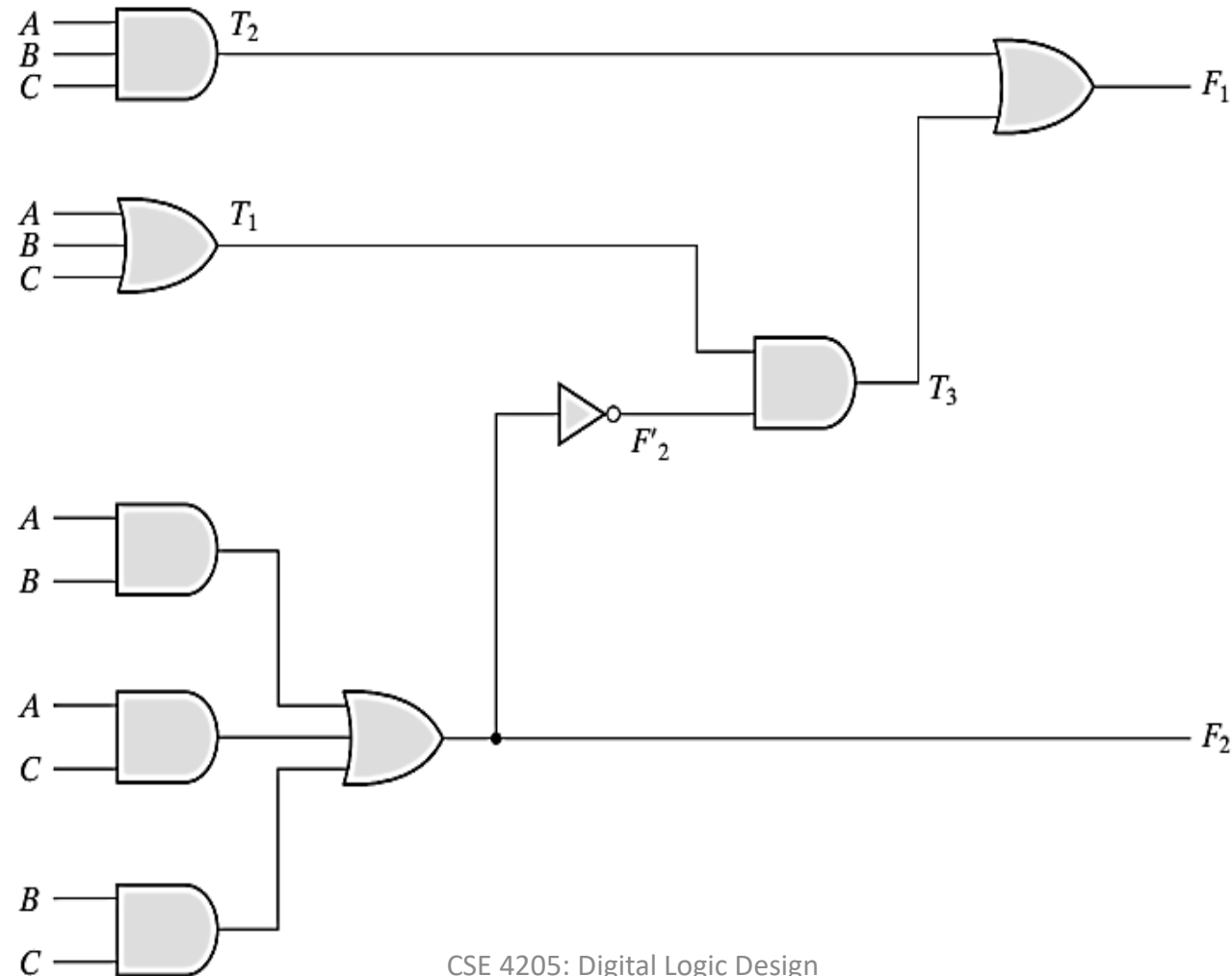


Analysis Procedure: To Obtain Boolean Function

1. Make sure that the given circuit is a combinational circuit, not sequential
2. To obtain Boolean function:
 - a) Label all the gate outputs that are the function of input variables with arbitrary symbols
 - Determine those functions
 - b) Also label the intermediate gate outputs that are the function of any input variables or previously defined symbols (*i.e.* already labeled gate outputs)
 - Find those functions as well
 - c) Repeat the process of step (b) until the output of the logic circuit is obtained
 - d) By repeated substitution of previously defined intermediate functions, obtain the output Boolean function in terms of input variables

AP: To Obtain a Boolean Function – Example

Given Logic Circuit





AP: To Obtain a Boolean Function – Example...

Following procedure

- Step (a):

$$F_2 = AB + AC + BC$$

$$T_1 = A + B + C$$

$$T_2 = ABC$$

- Step (b and c):

$$T_3 = F_2' T_1$$

$$F_1 = T_3 + T_2$$

- Step (d):

$$\begin{aligned} F_1 &= T_3 + T_2 = F_2' T_1 + ABC = (AB + AC + BC)'(A + B + C) + ABC \\ &= (A' + B')(A' + C')(B' + C')(A + B + C) + ABC \\ &= (A' + B'C')(AB' + AC' + BC' + B'C) + ABC \end{aligned}$$

$$F_1 = A'BC' + A'B'C + AB'C' + ABC$$



Analysis Procedure: To Obtain Truth Table

1. Make sure that the given circuit is a combinational circuit, not sequential
2. To obtain the truth table from the logic diagram directly:
 - a) Determine the number of input variables in the circuit
 - For n inputs, form the 2^n possible input combinations and list them as binary number from 0 to (2^n-1)
 - b) Label the intermediate gate outputs that are the function of any input variables or previously labeled gate outputs
 - c) Obtain the truth table for those intermediate outputs
 - d) Proceed to obtain for the outputs of those gates which are the function of previously defined values until the columns for all outputs are determined
3. Otherwise, from the derived function, it can be obtained very easily

AP: To Obtain Truth Table - Example

A	B	C	F₂	F'₂	T₁	T₂	T₃	F₁
0	0	0	0	1	0	0	0	0
0	0	1	0	1	1	0	1	1
0	1	0	0	1	1	0	1	1
0	1	1	1	0	1	0	0	0
1	0	0	0	1	1	0	1	1
1	0	1	1	0	1	0	0	0
1	1	0	1	0	1	0	0	0
1	1	1	1	0	1	1	0	1

Note: This truth table is identical to the truth table of the full adder



Design of Any Combinational Circuit

- Design of a combinational **circuit starts from a problem statement (design objective)**
 - And it **ends in a logic circuit diagram or a set of Boolean functions** from where the logic diagram can be obtained.
- There are some steps to be followed to achieve the requested combinational circuit



Design Procedure

1. From the specification of the circuit, determine the required number of inputs and outputs
 - Assign a symbol to each input and output variables
2. Derive the truth table from the specification that defines the required relationship between inputs and outputs
 - Input columns have 2^n different combinations for n input variables
 - Output values are determined from the problem statement
 - The specifications may indicate that some input combinations will not occur – **don't care conditions**
3. Obtain the simplified Boolean functions for each output as a function of the input variables
 - These output functions gives the exact definition of the circuit
 - **These output functions** should be **simplified by any simplification method** like k-map, tabular method, algebraic manipulation
4. Draw the logic diagram and verify the correctness of the design

Design Procedure: Considerations

- In a particular application, certain criteria will serve as a guide to choose an implementation among different alternatives
- A practical design method considers such constraints:
 - Minimum number of **gates**
 - Minimum number of **inputs to a gate**
 - Minimum **propagation time of the signal** through the circuit
 - Minimum number of **interconnections**
 - Limitations of the **driving capabilities of each gate** (*i.e.* the number of gates to which this output may be connected)
- Based on the application, those constraints will be prioritized.
- So there is no general statement to constitute an acceptable implementation



Design Procedure: Considerations...

- Most of the cases, the implementation begins with producing the simplified Boolean functions in standard form
 - It may proceed to further steps to meet other performance criteria

Design Procedure: Example

- **Circuit specification:**

- In practical different digital systems may use different codes and they may need to exchange information
- So a code converter circuit is required between them to make these two systems compatible even though they use different binary codes

- **Initial Consideration:**

- To convert from binary code A to binary code B, the input lines must have the bit combinations specified by code A
 - The output lines must generate the corresponding bit combinations specified by code B
 - Now we have to design this combinational circuit assuming code A as BCD and code B as excess-3 for decimal digits

Design Procedure: Example...

- **Truth Table** for 4 input and 4 output variables:

Input BCD				Output Excess-3 Code			
<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>w</i>	<i>x</i>	<i>y</i>	<i>z</i>
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0

Design Procedure: Example...

- **K-map** for 4 output variables to obtain their simplified Boolean functions:

		C			
		00	01	11	10
A	00	m_0 1	m_1	m_3	m_2 1
	01	m_4 1	m_5	m_7	m_6 1
	11	m_{12} X	m_{13} X	m_{15} X	m_{14} X
	10	m_8 1	m_9	m_{11} X	m_{10} X

$z = D'$

		C			
		00	01	11	10
A	00	m_0 1	m_1	m_3 1	m_2
	01	m_4 1	m_5	m_7 1	m_6
	11	m_{12} X	m_{13} X	m_{15} X	m_{14} X
	10	m_8 1	m_9	m_{11} X	m_{10} X

$y = CD + C'D'$

		C			
		00	01	11	10
A	00	m_0	m_1 1	m_3 1	m_2 1
	01	m_4 1	m_5	m_7	m_6 1
	11	m_{12} X	m_{13} X	m_{15} X	m_{14} X
	10	m_8	m_9 1	m_{11} X	m_{10} X

$x = B'C + B'D + BC'D'$

		C			
		00	01	11	10
A	00	m_0	m_1	m_3	m_2
	01	m_4	m_5 1	m_7 1	m_6 1
	11	m_{12} X	m_{13} X	m_{15} X	m_{14} X
	10	m_8 1	m_9 1	m_{11} X	m_{10} X

$w = A + BC + BD$

Design Procedure: Example...

- Two level logic diagrams of each output variables can be produced directly from their respective simplified expression in SOP forms
 - But with multiple-output system, these expressions can be manipulated algebraically further to obtain flexibility of common gates
 - Here, gate implementation might require three or more level of gates

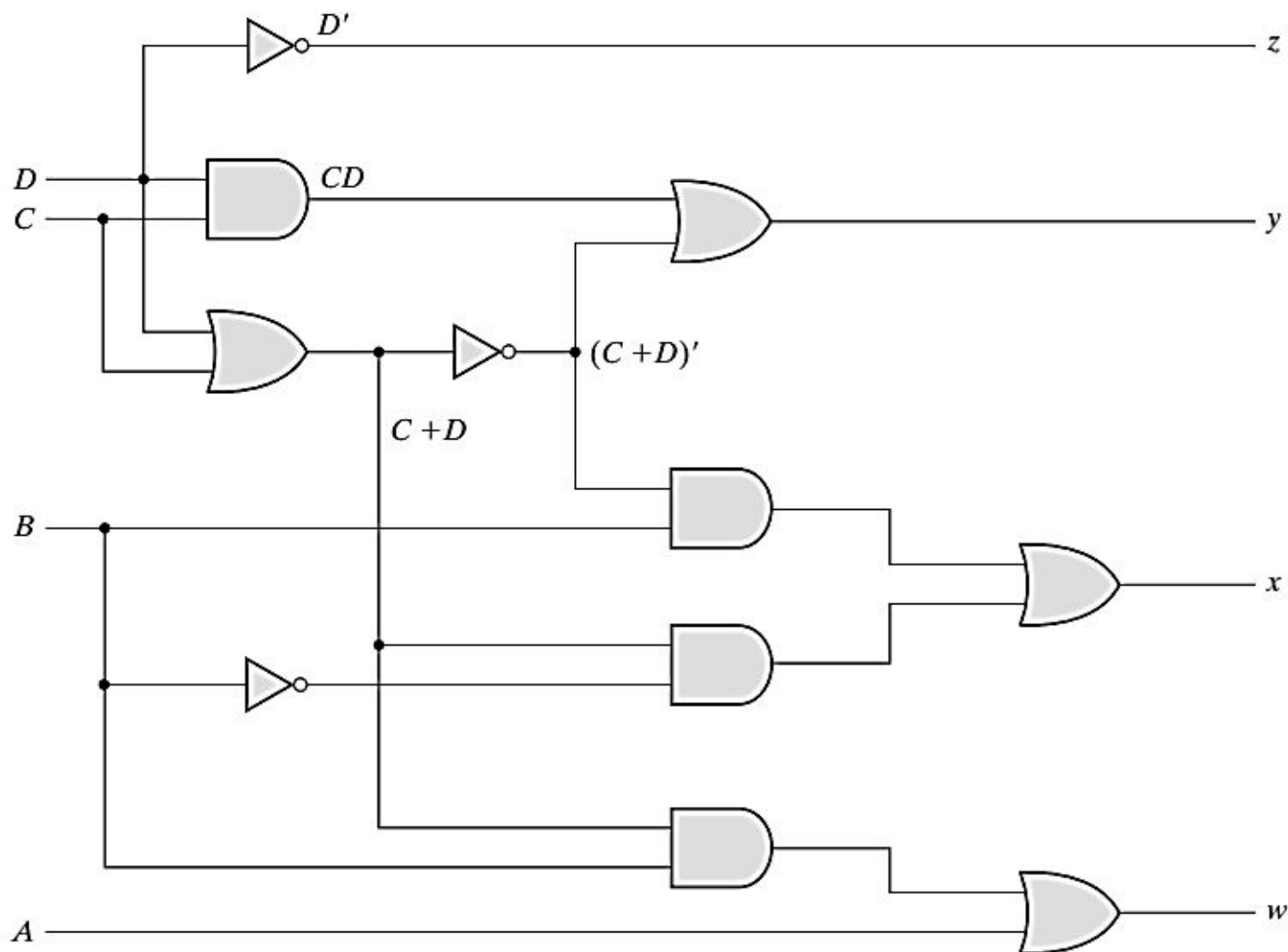
$$z = D'$$

$$y = CD + C'D' = CD + (C + D)'$$

$$\begin{aligned} x &= B'C + B'D + BC'D' = B'(C + D) + BC'D' \\ &= B'(C + D) + B(C + D)' \end{aligned}$$

$$w = A + BC + BD = A + B(C + D)$$

Design Procedure: Example – Logic Diagram



Binary Adder

- **Addition** – most common and basic arithmetic operation two binary digits
 - Four possible operations: $0+0=0$, $0+1=1$, $1+0=1$, and $1+1=10$
 - Carry is produced in fourth operation which may be propagated to the next significant digits

a	b	sum
0	0	0
1	0	1
0	1	1
1	1	10

- **Half Adder** – a combinational circuit that performs addition of two bits
- **Full Adder** – a combinational circuit that performs addition of three bits (*i.e.* two significant bits and a previous carry)
 - Two half adders can be employed to implement a full adder

Half Adder

- Two binary input bits (augend and addend) and two binary output bits (sum and carry)
 - Two input and two output variables
- **Assignment of symbols** – two inputs [x & y] and two outputs [S & C]
 - S represents the LSB and C represents the MSB of the sum
- Truth table to identify the function of the half adder:

Half Adder

x	y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

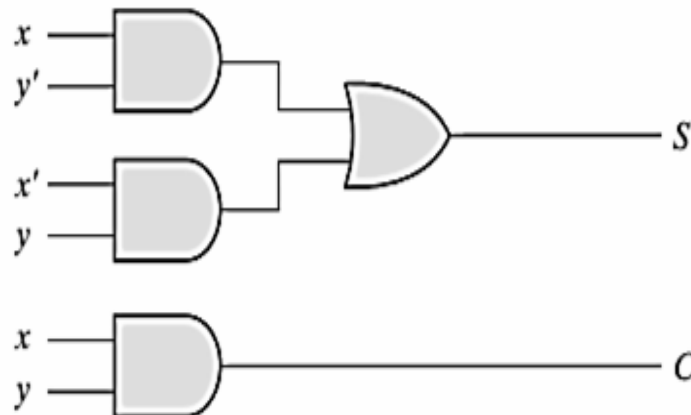
Half Adder – Function and Circuit

- Simplified **Boolean functions** for two outputs:

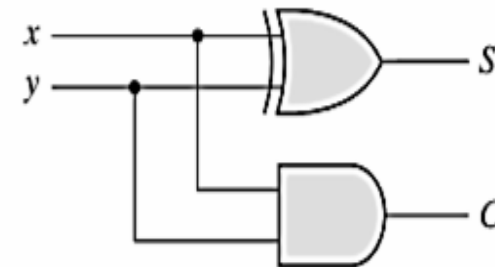
$$S = x'y + xy'$$

$$C = xy$$

- These functions could be implemented in different ways as well.
- **Circuit implementation** of half adder:



(a) $S = xy' + x'y$
 $C = xy$



(b) $S = x \oplus y$
 $C = xy$



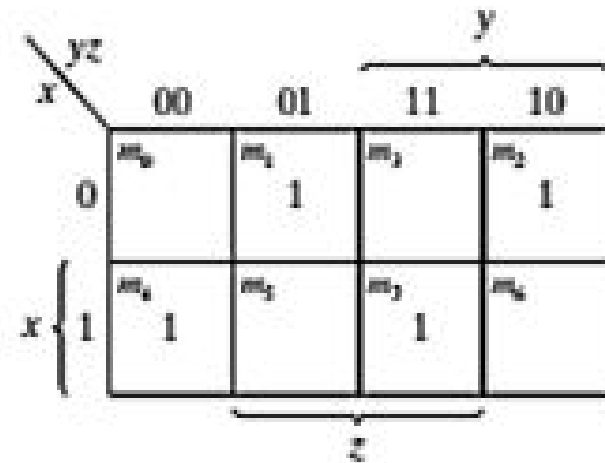
Full Adder

- For addition of n-bit binary numbers, it requires the use of a full adder
 - Bit by bit addition will be occurred beginning from the LSB
- A full adder is a **combinational circuit** that performs the **arithmetic sum of three input bits**
- Three inputs (x , y and z) and two outputs (S and C)
 - x and y are **two significant bits to be added**
 - z represents the **carry** from the **previous lower significant position**
 - Two outputs are necessary because the arithmetic sum of three binary digits ranges in value from 0 to 3
 - S is the LSB of the sum and C is the MSB

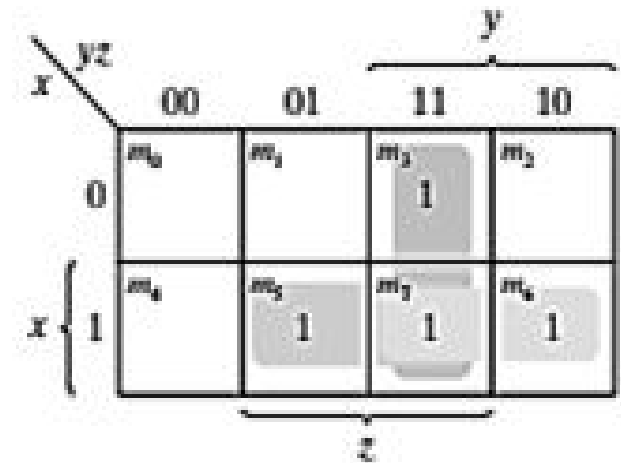
Full Adder – Truth table and K-map

Full Adder

x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

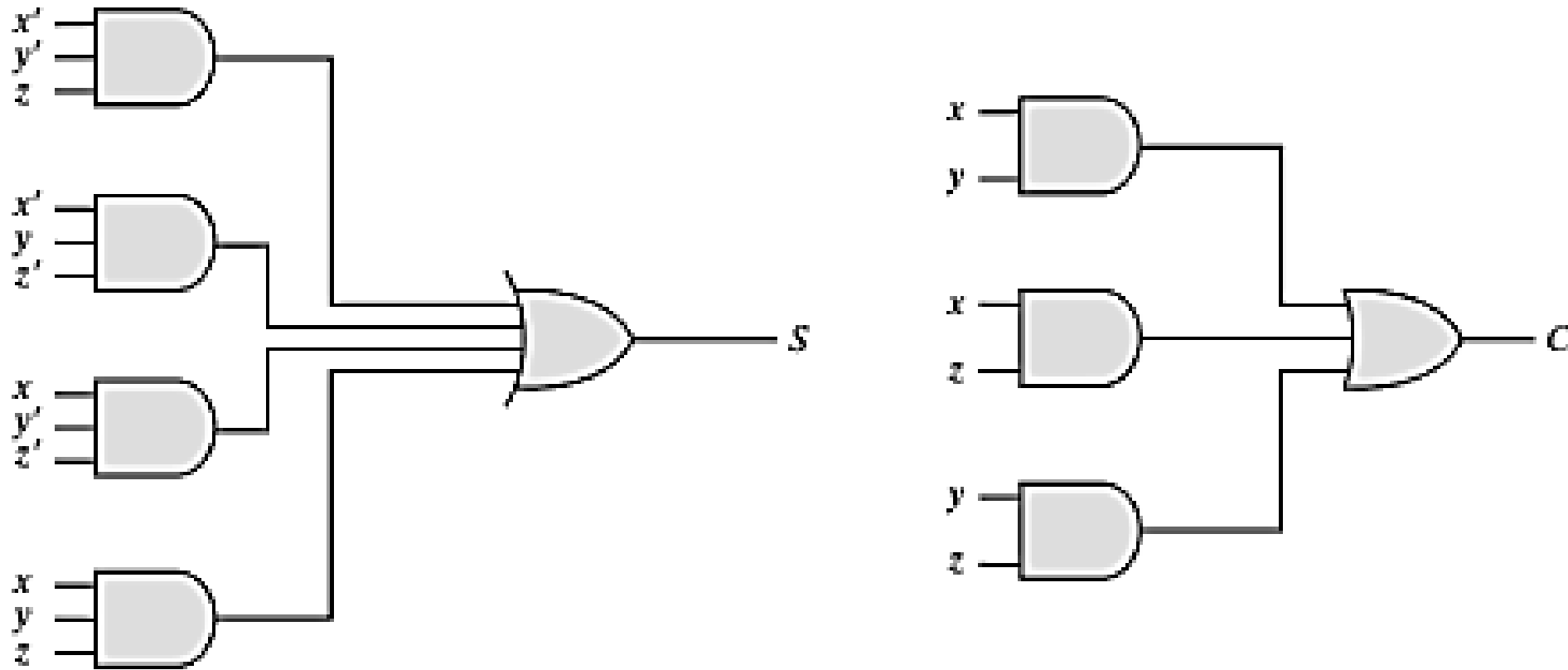


(a) $S = x'y'z + x'yz' + xy'z' + xyz'$

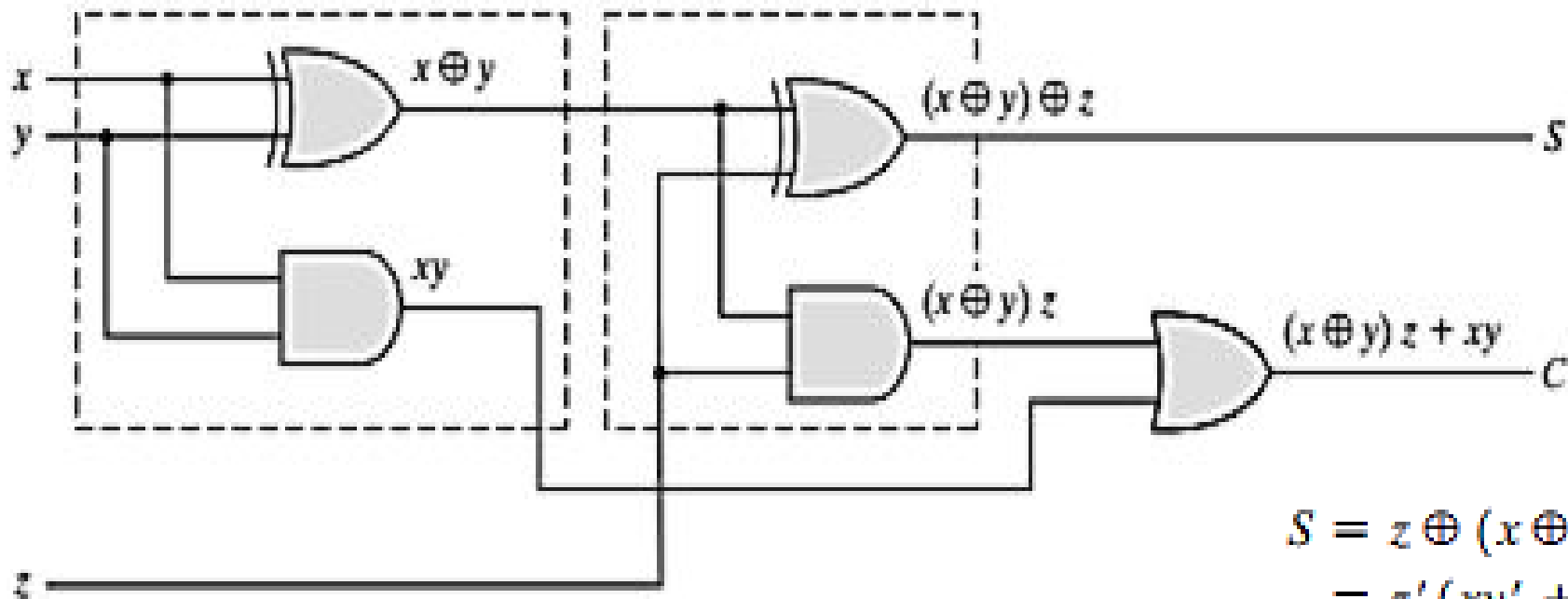


(b) $C = xy + xz + yz$

Full Adder – Implementation in SOP form



Full Adder – Implementation with Half Adder



$$\begin{aligned}
 S &= z \oplus (x \oplus y) \\
 &= z'(xy' + x'y) + z(xy' + x'y)' \\
 &= z'(xy' + x'y) + z(xy + x'y') \\
 &= xy'z' + x'yz' + xyz + x'y'z
 \end{aligned}$$

$$C = z(xy' + x'y) + xy = xy'z + x'yz + xy$$

Binary Adder

Binary Ripple Carry Adder

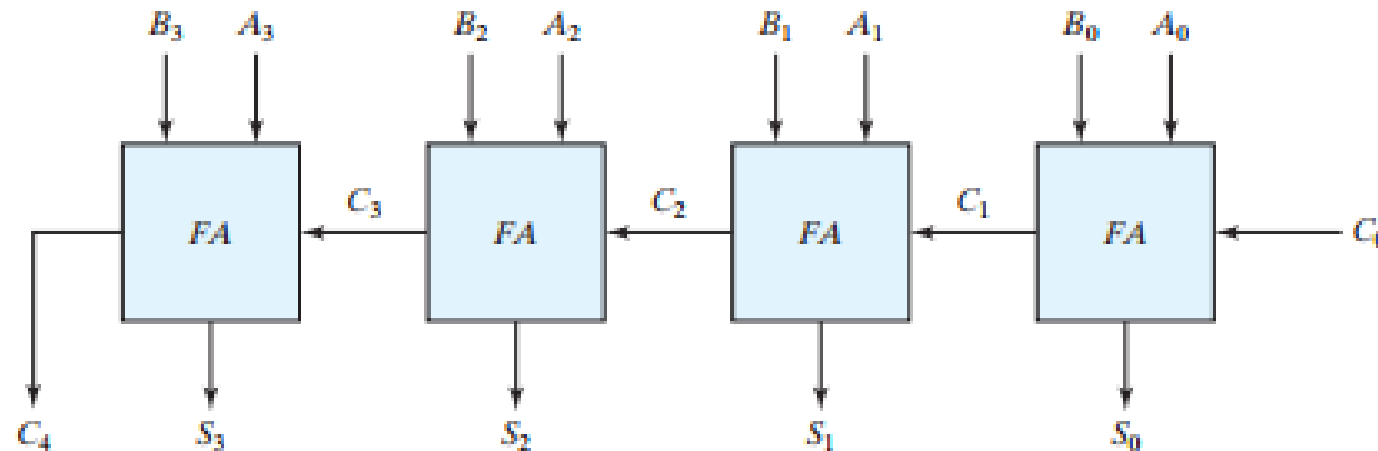
- **Binary adder** – a digital circuit that produces the arithmetic sum of **two binary numbers** of n number of bits
 - It requires a chain of n **full adders** or a chain of one **half adder** and $(n - 1)$ **full adders** in cascade.
 - Subscript 0 denotes the LSB and **($n-1$)** denotes the MSB
 - The input carry C_0 in the least significant position of the first case must be 0.
 - Output carry of any full adder is connected to the input carry of the next full adder in the chain
 - Example: $A=1011$ and $B=0011$ will produce the sum, $S=1110$ with four bit adder as follows:

Subscript i :	3	2	1	0	
Input carry	0	1	1	0	C_i
Augend	1	0	1	1	A_i
Addend	0	0	1	1	B_i
Sum	1	1	1	0	S_i
Output carry	0	0	1	1	C_{i+1}

Binary Adder

Binary Ripple Carry Adder

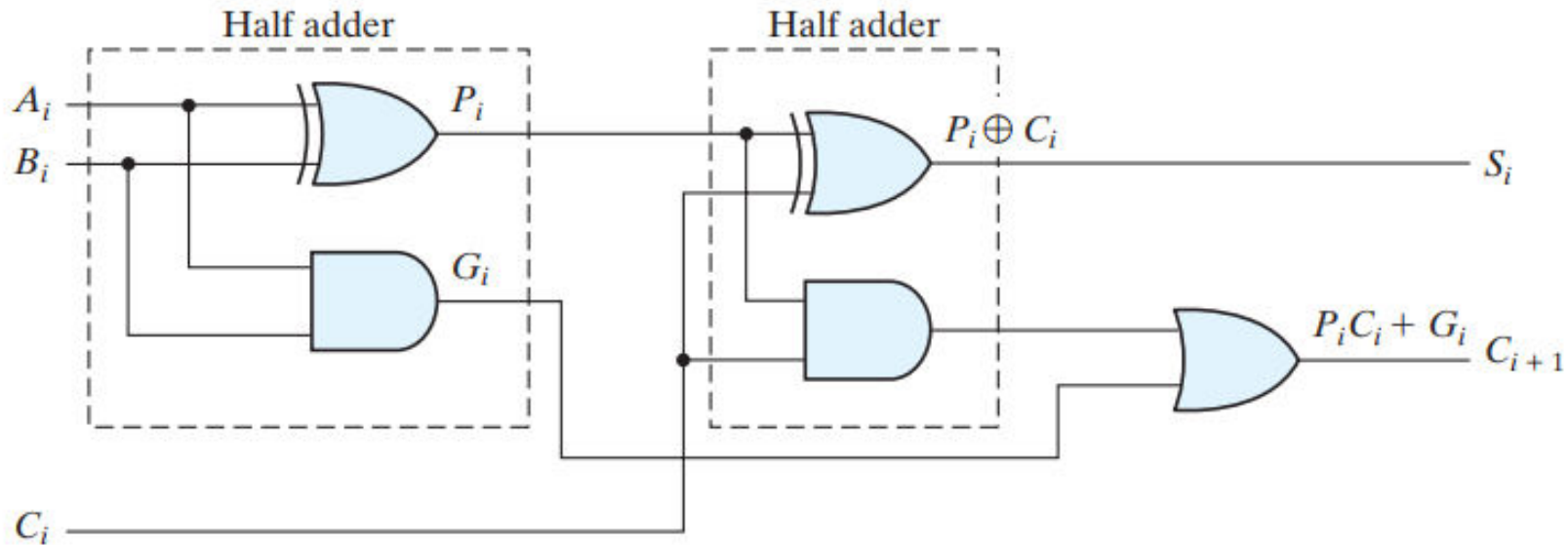
- If we follow the classical method, it would require a truth table with $2^9=512$ entries
 - By using an **iterative method** of **cascading a standard function**, it is possible to obtain a simple and straightforward implementation.
 - All the carries must be generated to obtain correct sum at the outputs.



Carry Propagation

- As any combinational circuit, the signal must be propagated through the gates before the correct outputs are present in the output terminals
 - ***Total propagation delay = Delay in a gate $\times$ Number of gate levels***
- Any combinational circuit will always have some value at its outputs
 - These outputs will not be correct unless the enough propagation delay is maintained
- This propagation delay is crucial if a system uses this type of adder successively
 - Solution:
 - Faster gates in this circuit – **but** impractical
 - Complex circuit – **carry lookahead logic**

Carry Propagation with Full Adder



$$P_i = A_i \oplus B_i$$

$$G_i = A_i B_i$$

$$S_i = P_i \oplus C_i$$

$$C_{i+1} = G_i + P_i C_i$$

Carry Propagation with 4-bit Full Adder

To generate Carry Lookahead

$$C_0 = \text{input carry}$$

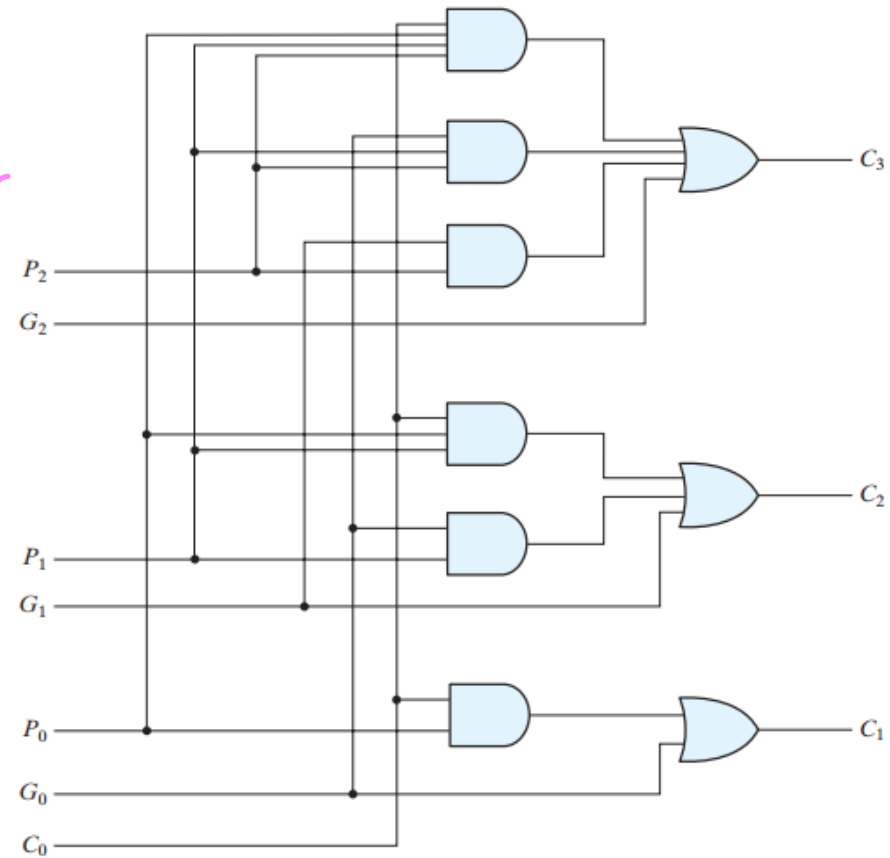
$$C_1 = G_0 + P_0C_0$$

$$C_2 = G_1 + P_1C_1 = G_1 + P_1(G_0 + P_0C_0) = G_1 + P_1G_0 + P_1P_0C_0$$

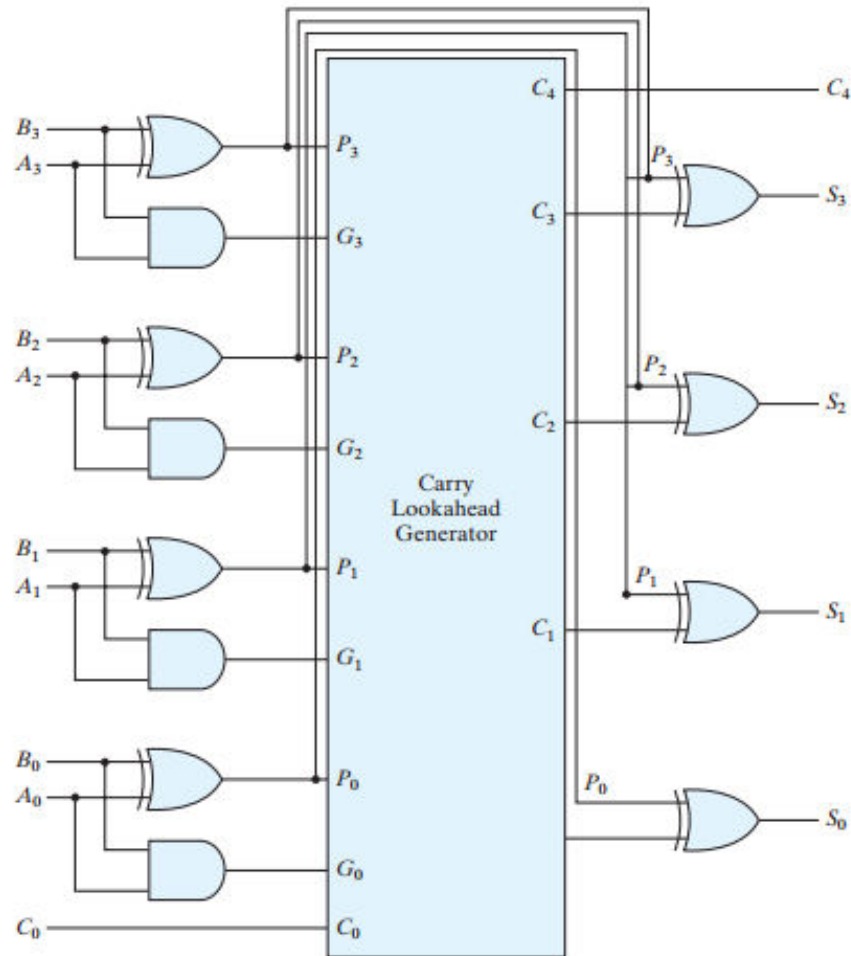
$$C_3 = G_2 + P_2C_2 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0$$

Carry Lookahead Generator Circuit

For 4-bit Adder



4-bit Binary Adder with Carry Lookahead





Binary Subtractor

- The subtraction of two unsigned binary numbers – *taking the complement of the subtrahend* and adding it to the minuend
 - Complement of a number can be obtained by 1's or 2's complement
- So subtraction is a kind of addition operation – needs **full adder** for machine implementation
- If the minuend bit is smaller than the subtrahend bit, a 1 is borrowed from the next significant position – *also must be conveyed to the next higher pair of bits by means of a binary signal from current stage to next higher stage*



Binary Subtractor...

- Two kind of subtractor based on the number of inputs:
 - Half-subtractor
 - Full subtractor
- For **signed-complement system**, addition and subtraction could be implemented using same circuit like unsigned number system.
 - Users should **interpret** the system wisely considering it is signed or unsigned number
 - So **same circuit** could be used to handle both types of arithmetic

Half-Subtractor

- A combinational circuit – subtracts two bits and produces **their differences** and **an output** that specifies if a 1 is **borrowed** or not to next stage.
- Designation of the input variables – minuend (x) and subtrahend (y)
- $D = 2B + x - y$, [if $x < y$, $B=1$]

Inputs		Outputs	
X	Y	D	B
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

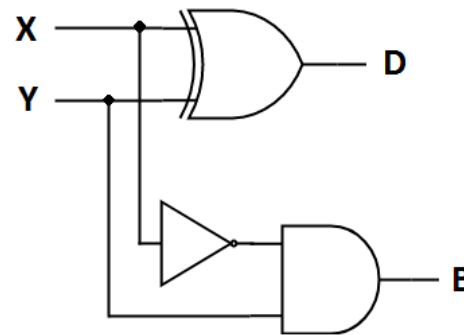
Half-Subtractor...

- Boolean functions of two outputs of the half-subtractor:

$$D = x'y + xy'$$

$$B = x'y$$

- Logic for D is same as the logic for S in half-adder
- Circuit Implementation of half-subtractor:





Full-Subtractor

- A combinational circuit – performs a subtraction between two bits considering the **previous borrow** – three inputs and two outputs
- Designation of variables – inputs [minuend(x), subtrahend(y), previous borrow(z)] and outputs [difference(D) and current borrow (B)]
- Considering $D = 2B + x - y - z$, if $z=0$, it will be same as half-subtractor
 - If $(x < y + z)$, $B=1$

Full-Subtractor...

- Truth table of the full-subtractor:

X	Y	Z	D	B
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

- K-map for full-subtractor to simplify:

		yz			
		00	01	11	10
x	0	0	1	1	1
	1	0	0	1	0

For Borrow

		yz			
		00	01	11	10
x	0	0	1	0	1
	1	1	0	1	0

For Difference



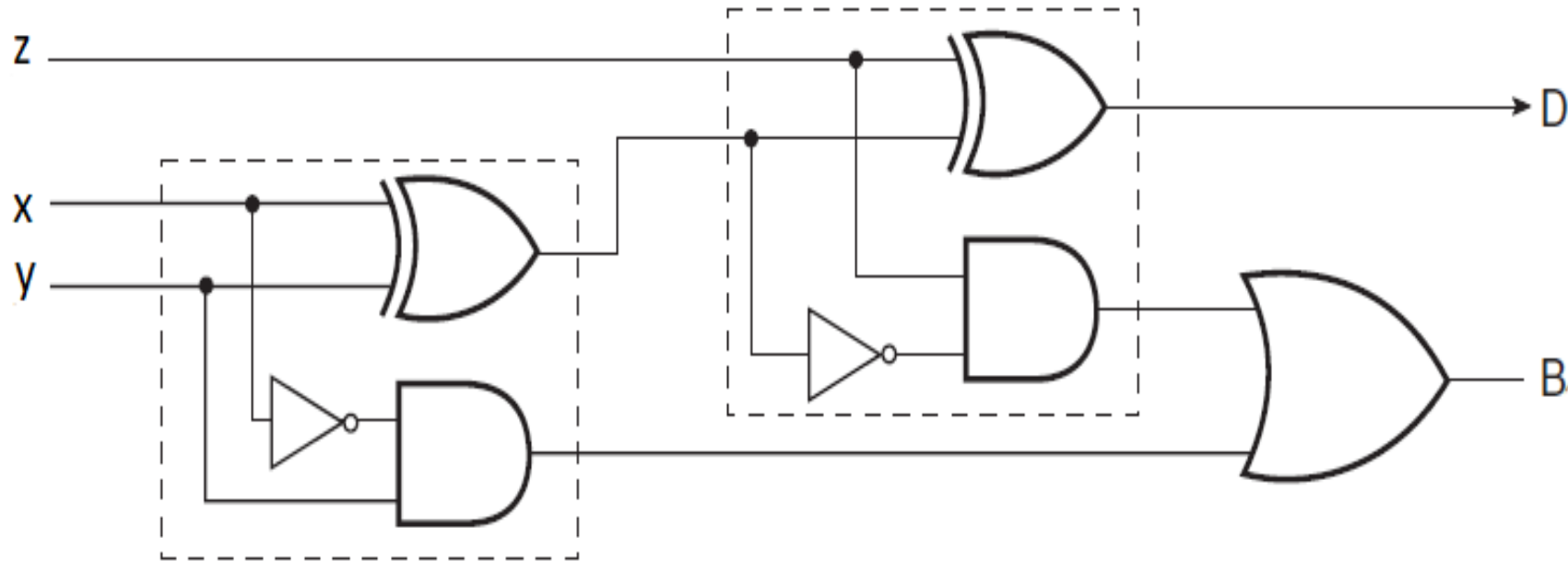
Full-Subtractor...

- The simplified Boolean function of full-subtractor:

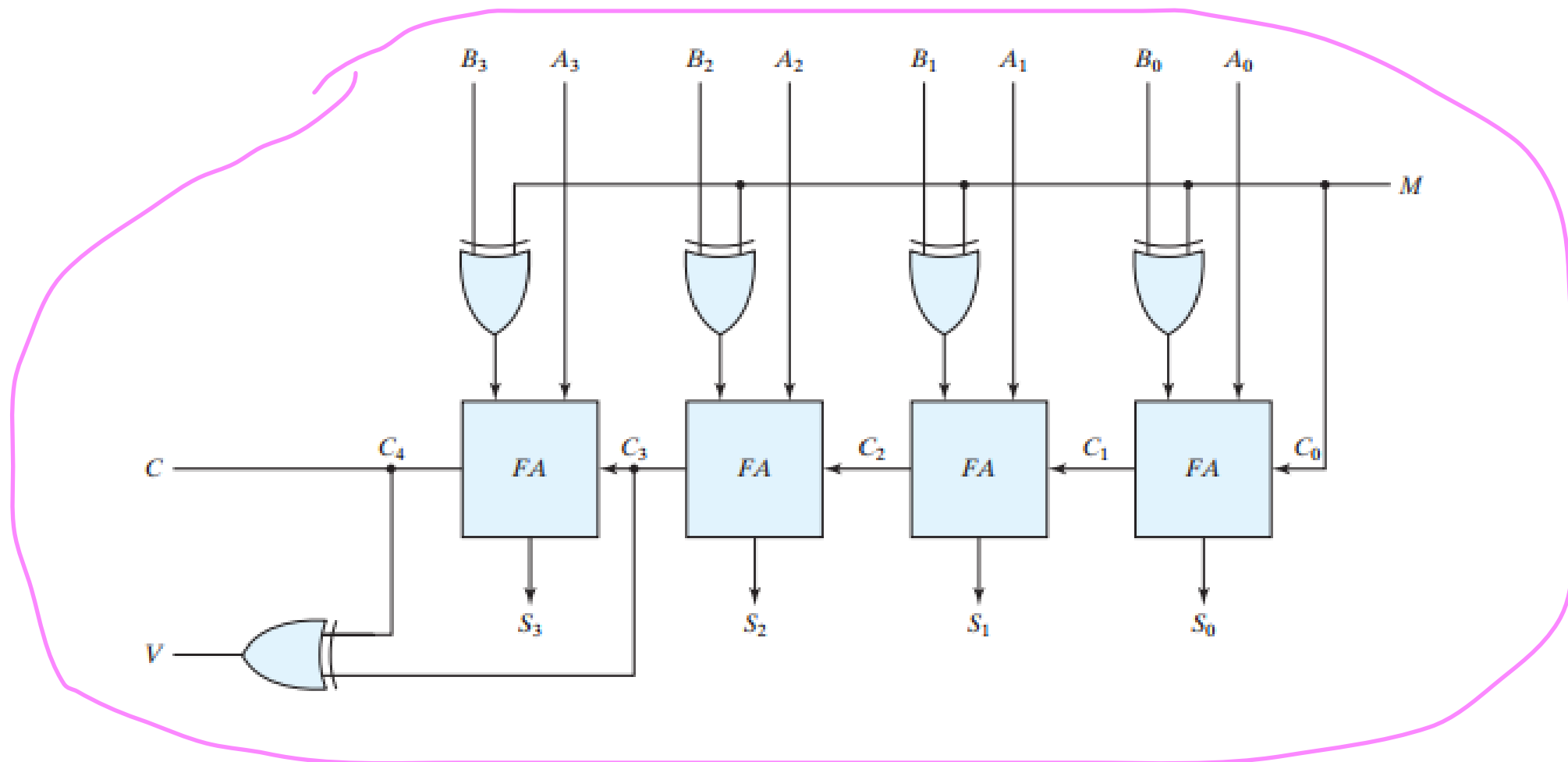
$$D = x'y'z + x'yz' + xy'z' + xyz$$
$$B = x'y + x'z + yz$$

- The output of **B** resembles the function for **C** in the full adder – **except x is complemented**
- The output of **D** is exactly same as the output **S** in the full adder
- Because of these similarities, it is possible to convert a full adder into a full subtractor - ???

Full-Subtractor with Half-Subtractors



4-bit Full Adder-Subtractor



Overflow

- When two numbers with n digits each are added and the sum is a number occupying $n + 1$ digits
 - Problem in digital computers, as there is a finite number of bits
 - Overflow detection is required to handle this occurrence
 - Not possible with paper and pencil as there is no limit of space
- **Only happens during addition**, but not for subtraction
 - When both numbers are either positive or negative
- Also depends on the types of numbers:
 - **Unsigned numbers:**
 - Overflow detected by **end carry of the MSB**
 - **Signed numbers :**
 - **A range is maintained from max positive to min negative. If the sum crosses the range, an overflow occurs**
 - **(n+1)th bit indicates the sign bit of the result**
 - **Note:** Sign bit is treated as a part of a number and carry doesn't indicate an overflow

Overflow - Example

carries:	0 1
+70	0 1000110
+80	0 1010000
+150	1 0010110

carries:	1 0
−70	1 0111010
−80	1 0110000
−150	0 1101010

Note that the eight-bit result that should have been positive has a negative sign bit (i.e., the eighth bit) and the eight-bit result that should have been negative has a positive sign bit. As 9-bit storage is not possible, an overflow has occurred.

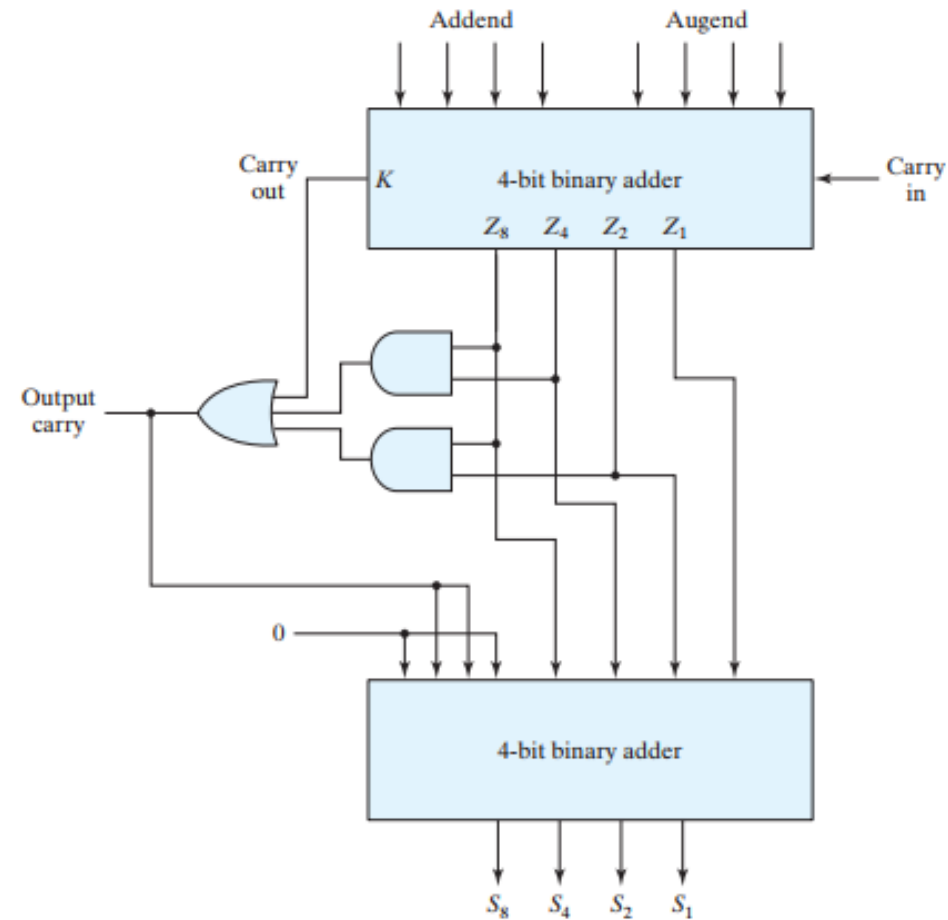
Overflow condition: carry in and out in the MSB position is not same. (Try +70 and +50 to check overflow)

BCD Adder

Binary Sum					BCD Sum					Decimal
<i>K</i>	<i>Z</i> ₈	<i>Z</i> ₄	<i>Z</i> ₂	<i>Z</i> ₁	<i>C</i>	<i>S</i> ₈	<i>S</i> ₄	<i>S</i> ₂	<i>S</i> ₁	
0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	2
0	0	0	1	1	0	0	0	1	1	3
0	0	1	0	0	0	0	1	0	0	4
0	0	1	0	1	0	0	1	0	1	5
0	0	1	1	0	0	0	1	1	0	6
0	0	1	1	1	0	0	1	1	1	7
0	1	0	0	0	0	1	0	0	0	8
0	1	0	0	1	0	1	0	0	1	9
0	1	0	1	0	1	0	0	0	0	10
0	1	0	1	1	1	0	0	0	1	11
0	1	1	0	0	1	0	0	1	0	12
0	1	1	0	1	1	0	0	1	1	13
0	1	1	1	0	1	0	1	0	0	14
0	1	1	1	1	1	0	1	0	1	15
1	0	0	0	0	1	0	1	1	0	16
1	0	0	0	1	1	0	1	1	1	17
1	0	0	1	0	1	1	0	0	0	18
1	0	0	1	1	1	1	0	0	1	19

$$C = K + Z_8Z_4 + Z_8Z_2$$

BCD Adder – Block Diagram

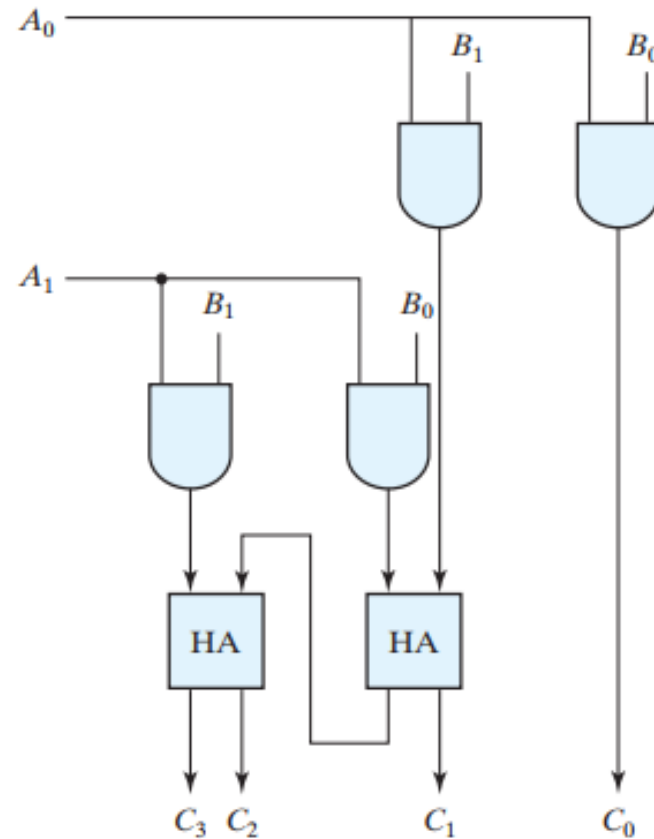


Binary Multiplier

- Multiplication in binary is performed in the same way as multiplication in decimal
 - Multiplicand is multiplied by the each bit of the multiplier starting from the LSB
 - Each such multiplication forms a partial product
 - Successive partial products are shifted one position to the left
 - Final product is obtained from the sum of the partial products
 - **Example:** If **A**, **B** and **C** are binary variables: $C = A \times B$

Multiplicand	B_1	B_0	
Multiplier	A_1	A_0	
	A_0B_1	A_0B_0	
	A_1B_1	A_1B_0	
C_3	C_2	C_1	C_0

Two Bit Binary Multiplier



Note: Multiplier with more bits can be constructed in a similar fashion



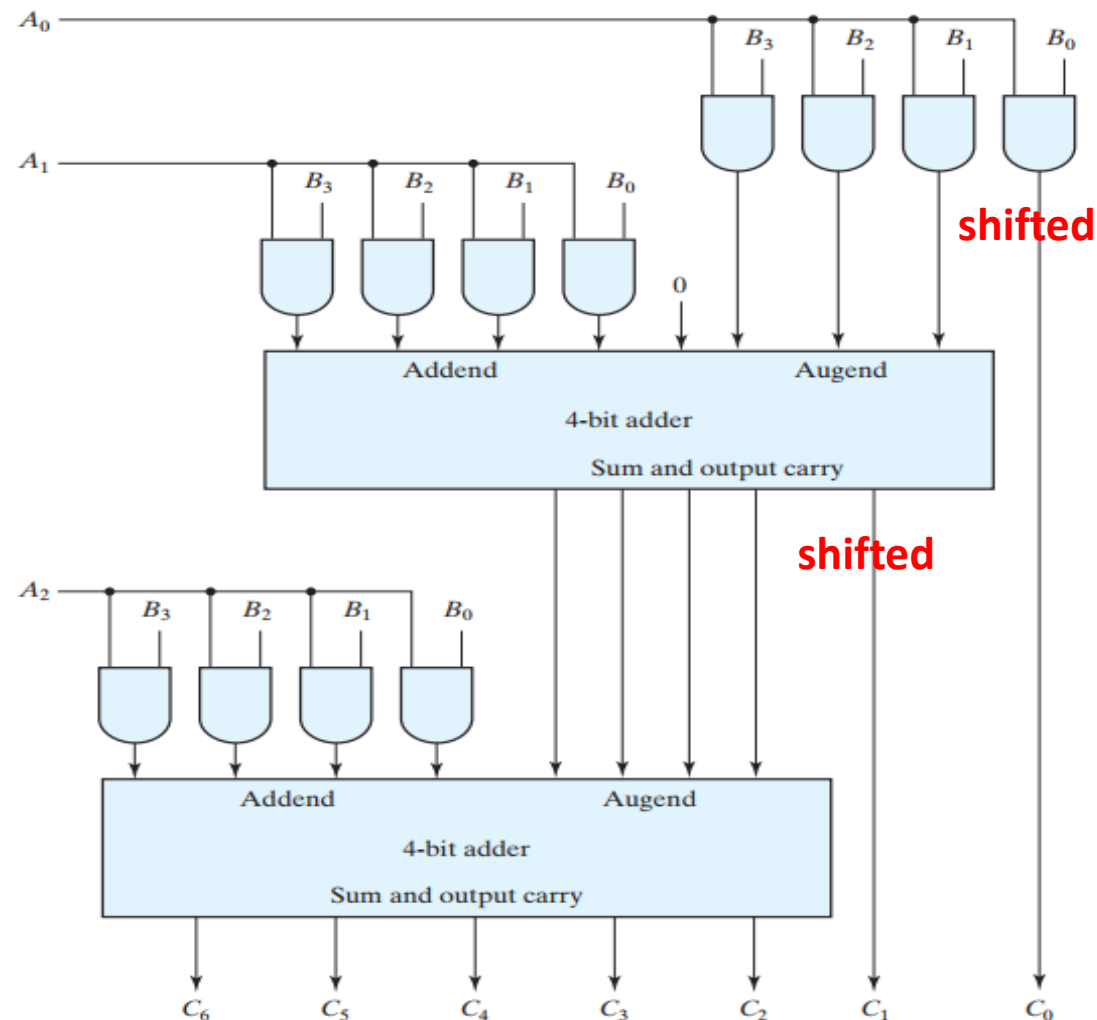
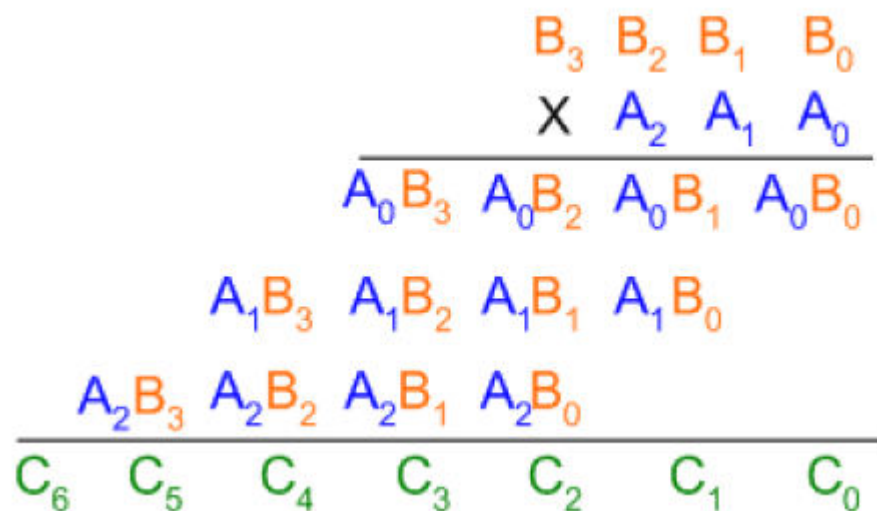
Binary Multiplier – Important

- The number of partial products (levels of AND gates) is same as the number of bits in the multiplier
- J multiplier bits and K multiplicand bits will require in multiplication:
 - $(J \times K)$ AND gates
 - $(J - 1)$ K -bit adders
- to produce a product of $(J + K)$ bits

Multiplicand	B_1	B_0	
Multiplier	A_1	A_0	
	A_0B_1	A_0B_0	
	A_1B_1	A_1B_0	
C_3	C_2	C_1	C_0

Example – Binary Multiplier

- Example: $C(?) = A(A_2A_1A_0) \times B(B_3B_2B_1B_0)$





Binary Magnitude Comparator

- Comparison operation determines between two numbers whether one number is greater than, less than or equal to the other number
- Binary magnitude comparator:
 - A combinational circuit that compares two binary numbers **A** and **B**
 - Outcome of the comparison is specified by three binary variables that indicate whether $A > B$, $A = B$ or $A < B$
- **Note:** If ordinary design procedure is followed, for two n-bit numbers, the truth table will have 2^{2n} entries
 - Too cumbersome to handle
 - Example: with $n=3$, the truth table will have 64 rows
- Rather a pattern is followed (an algorithm) to solve this problem
 - Some patterns occurs frequently which makes it easy build a simple combinational circuit.

Four Bit Magnitude Comparator

- Comparator with single bit:
 - If $A > B$
 - If $A = B$
 - If $A < B$
- Comparator with Four bits:
 - Try the above methods four times and add them up
- Expressions:

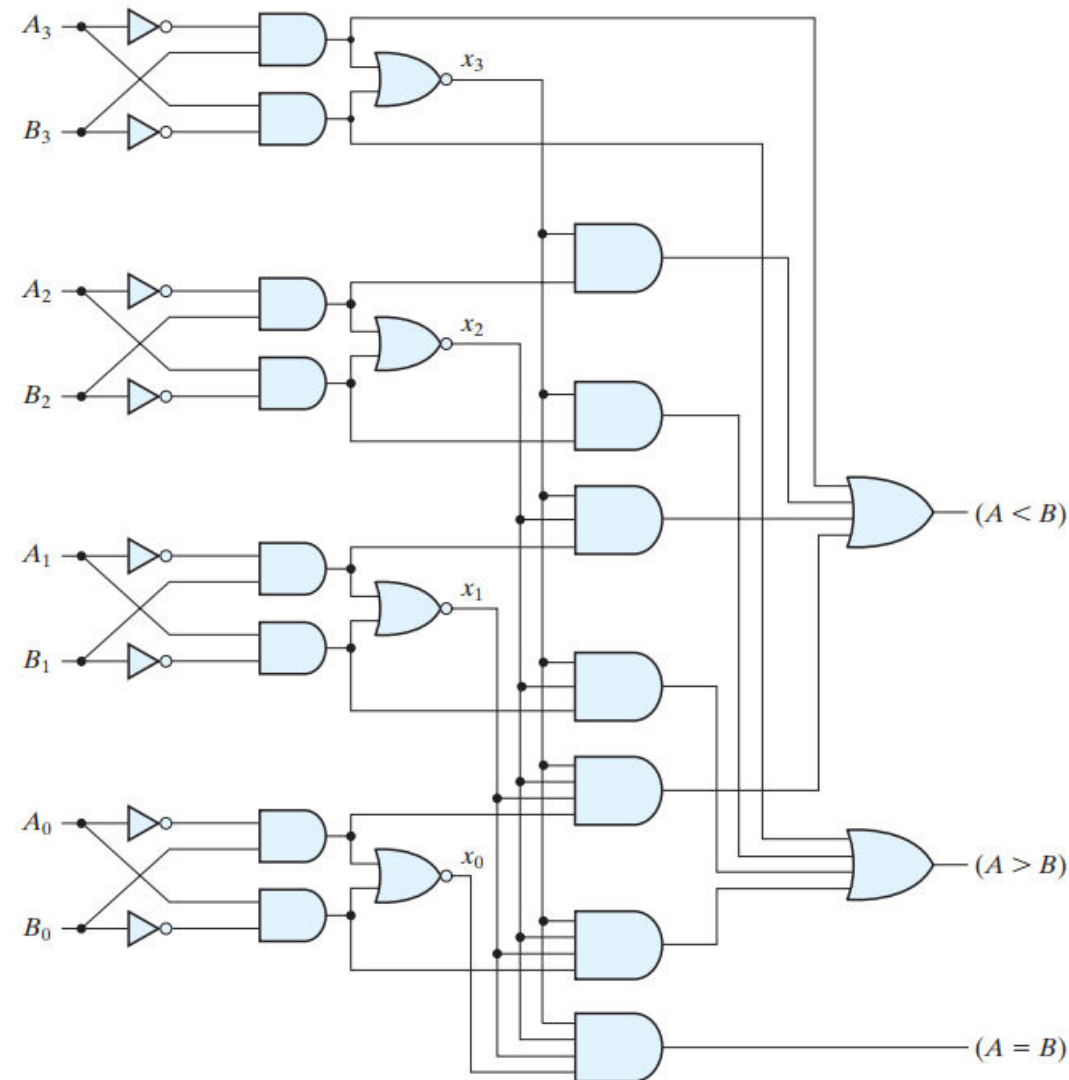
$$x_i = A_i B_i + A'_i B'_i \quad \text{for } i = 0, 1, 2, 3$$

$$(A = B) = x_3 x_2 x_1 x_0$$

$$(A > B) = A_3 B'_3 + x_3 A_2 B'_2 + x_3 x_2 A_1 B'_1 + x_3 x_2 x_1 A_0 B'_0$$

$$(A < B) = A'_3 B_3 + x_3 A'_2 B_2 + x_3 x_2 A'_1 B_1 + x_3 x_2 x_1 A'_0 B_0$$

Four Bit Magnitude Comparator Circuit





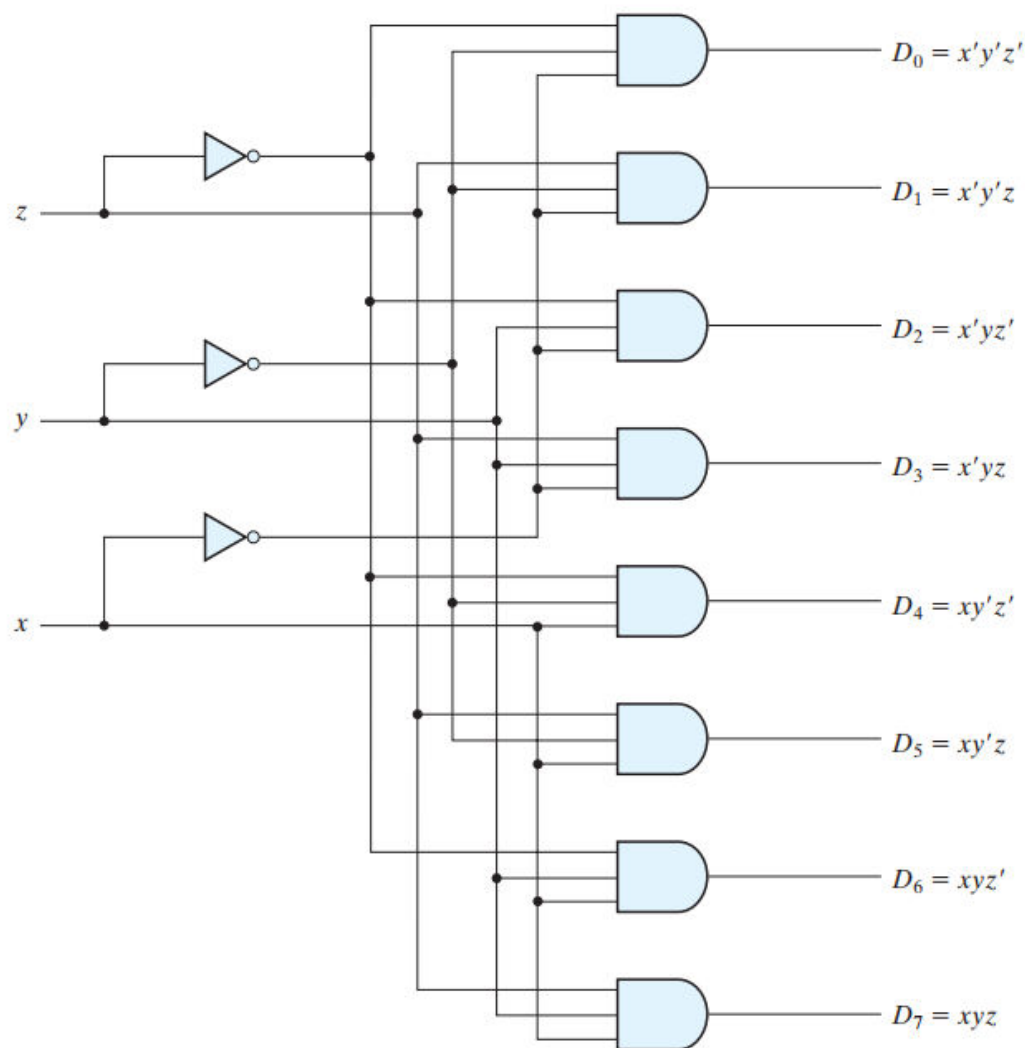
Encoder-Decoder

- Normally, any discrete information can be represented in binary code
 - Here, a n -bit binary code can represent 2^n different elements
- **Encoders** convert 2^n unique lines of input into **a code** of n bits and **Decoders** decode the n bits into 2^n unique lines.

Decoder

- For a decoder, if it has unused combinations for a n -bit code, it will produce the output lines less than 2^n
- **Representation of a decoder: *n-to-m line* decoder**
 - Where $m \leq 2^n$
- **Usage of decoder:**
 - To generate the 2^n (or fewer) **minterms** of *n* input variables
 - Moreover, this circuit could be used for code conversion (e.g. *binary to octal*)

3x8 Decoder: as a Binary to Octal Converter



Inputs			Outputs							
x	y	z	D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

Decoder with NAND Gate and Enable Input

- Some decoders are constructed with NAND gates to make it more economical
 - In this case, minterms are generated in the complemented form
 - Similarly, the operation of the decoder can be controlled with 0s instead of 1s.
- Some of them can include one or more **enable inputs** to control the circuit operations
 - **Variants:** Active-low and active-high enable

Active High inputs & outputs

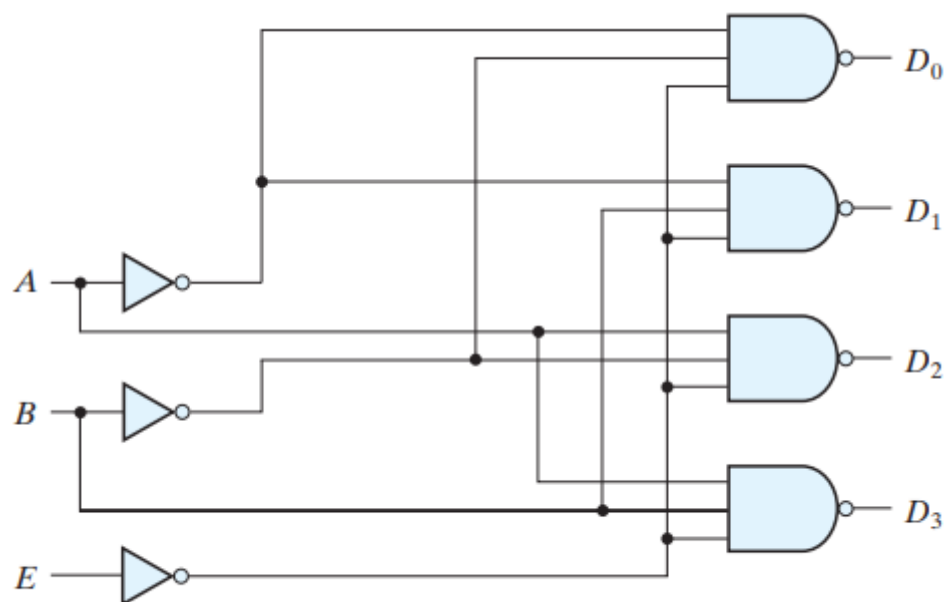


Active Low inputs & outputs



Decoder with NAND gates and Enable Input (E)

- 0 is considered “**enable**” for both inputs and E because of **NAND gate**



(a) Logic diagram

E	A	B	D_0	D_1	D_2	D_3
1	X	X	1	1	1	1
0	0	0	0	1	1	1
0	0	1	1	0	1	1
0	1	0	1	1	0	1
0	1	1	1	1	1	1

Specialty with “X” in inputs **condense** the truth table

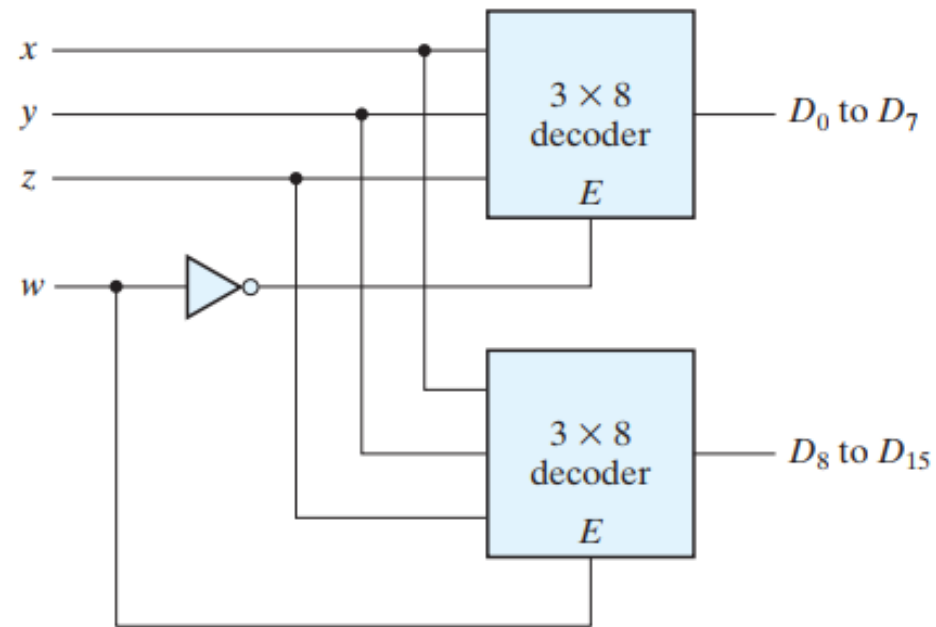


Demultiplexer with Decoder

- Discussed later on in this chapter
 - At first we have to know about multiplexer

Larger Decoder with Smaller Ones

- 4x16 decoder with two 3x8 decoders:





Significance of Enable Input

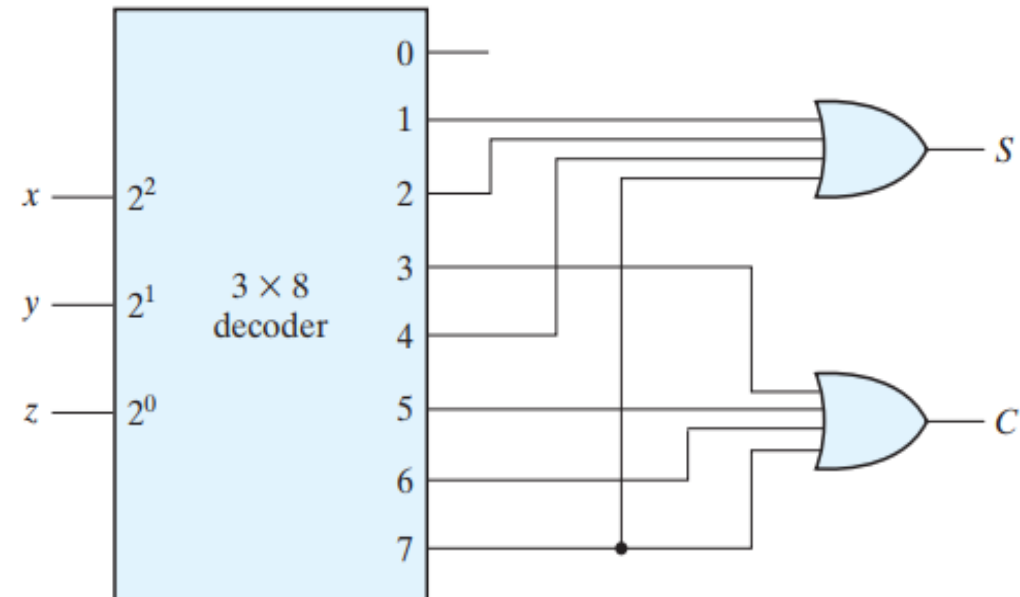
- In general, ***enable inputs*** are a convenient feature for interconnecting **two or more standard components** for the purpose of combining them into a similar function with more inputs and outputs.

Combinational Circuit with Decoders

- We consider the minterms of any function from the truth table of certain Boolean function and add them with an external OR gate for each output variable
- **Example:** Full Adder with 3x8 decoder

$$S(x, y, z) = \Sigma(1, 2, 4, 7)$$

$$C(x, y, z) = \Sigma(3, 5, 6, 7)$$





Combinational Circuit with Decoders...

- If the number of minterms (k) is so many ($>2^n/2$) we can consider the F' with (2^n-k) number of minterms
 - Then NOR gate will be used instead of OR gate
- If the internal gate of the Decoder is NAND gate, the external gate will be also a NAND gate instead of an OR gate
 - **Two level NAND** gate implementation



Encoder

- Inverse of Decoder
- 2^n inputs and n outputs
 - Only one input will be 1 at a time
- Another kind of converter or coder or cryptographer

8x3 Encoder: Truth Table and Circuit

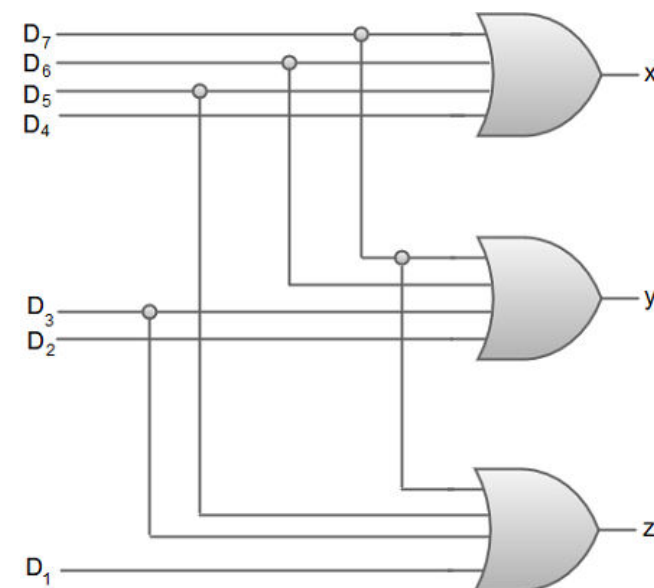
Octal to Binary Encoder

$$z = D_1 + D_3 + D_5 + D_7$$

$$y = D_2 + D_3 + D_6 + D_7$$

$$x = D_4 + D_5 + D_6 + D_7$$

Inputs								Outputs		
D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7	x	y	z
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	1	0	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	1	1	1	1



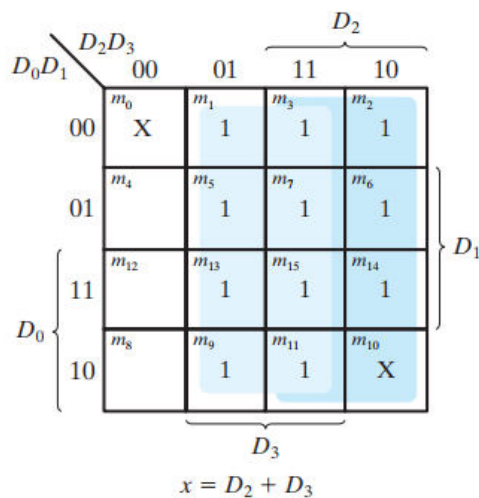
Problems: Ambiguity

1. If there are 1s in more than one inputs, output could be **undefined**
 - **Solution:** Priority encoder to ensure the input priority
2. 0 for two different representations – (i) when all the inputs are 0 and (ii) $D_0 = 1$
 - **Solution:** Add another output to observe

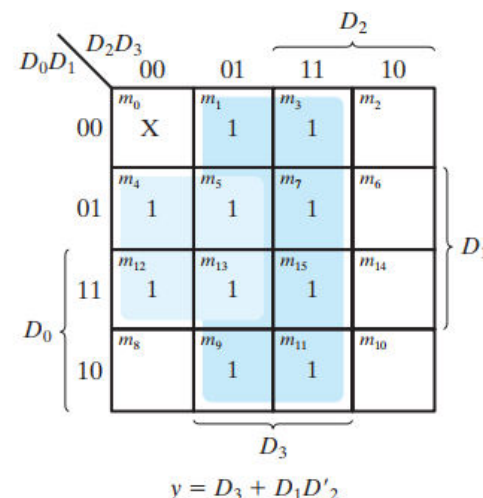
Priority Encoder with additional Output

- It includes the priority function for the inputs
- Truth table and K-map:**

Inputs				Outputs		
D_0	D_1	D_2	D_3	x	y	V
0	0	0	0	X	X	0
1	0	0	0	0	0	1
X	1	0	0	0	1	1
X	X	1	0	1	0	1
X	X	X	1	1	1	1



K-map for output x . The map shows 1s in cells $m_1, m_3, m_5, m_7, m_{13}, m_{15}$. The prime implicants are D_2 (covering m_1, m_3, m_5, m_7) and D_3 (covering m_5, m_7, m_{13}, m_{15}). The expression is $x = D_2 + D_3$.



K-map for output y . The map shows 1s in cells $m_1, m_3, m_5, m_7, m_{13}, m_{15}$. The prime implicants are D_3 (covering m_5, m_7, m_{13}, m_{15}) and $D_1 D'_2$ (covering m_1, m_3). The expression is $y = D_3 + D_1 D'_2$.

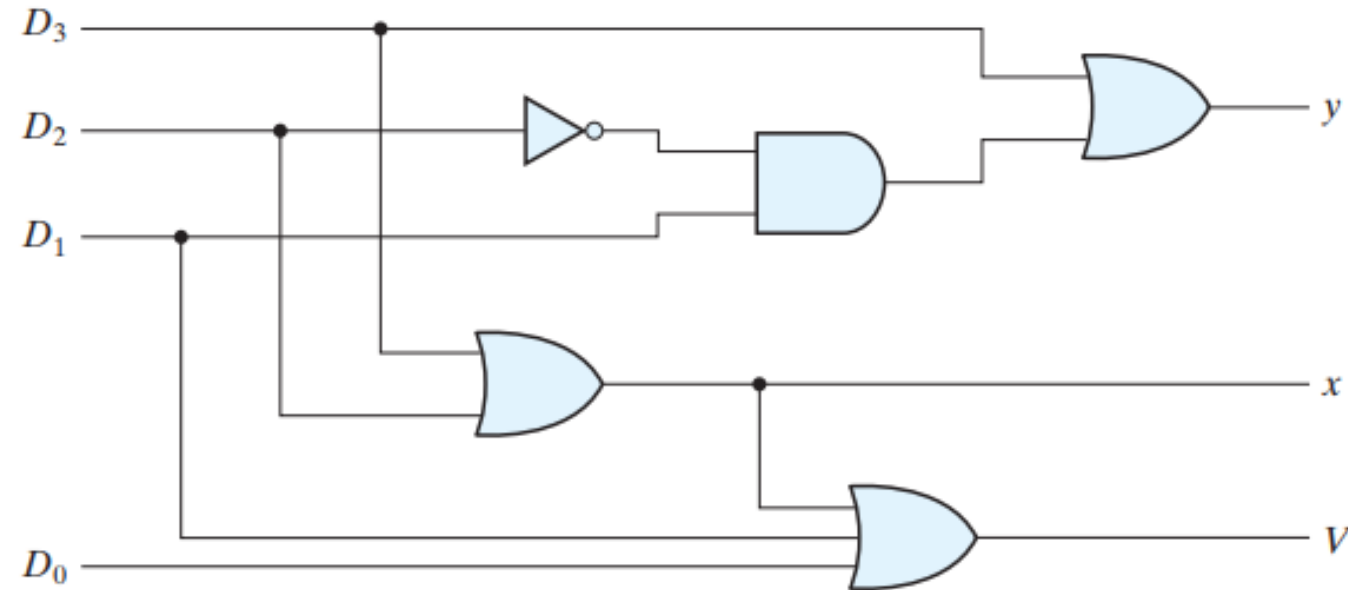
$$x = D_2 + D_3$$

$$y = D_3 + D_1 D'_2$$

$$V = D_0 + D_1 + D_2 + D_3$$

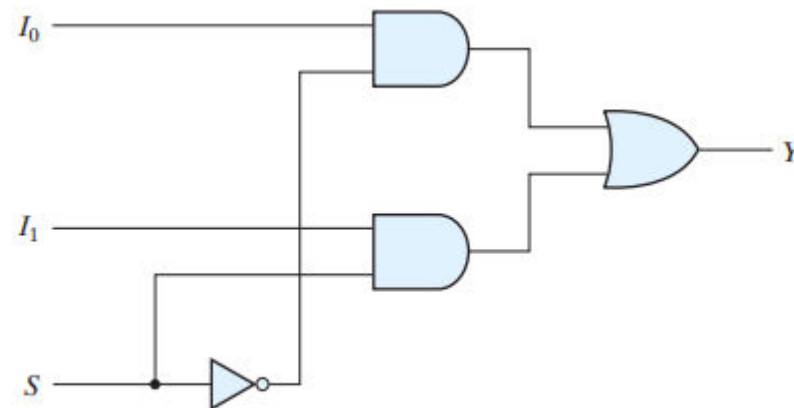
Priority Encoder with additional Output...

- **Circuit:**

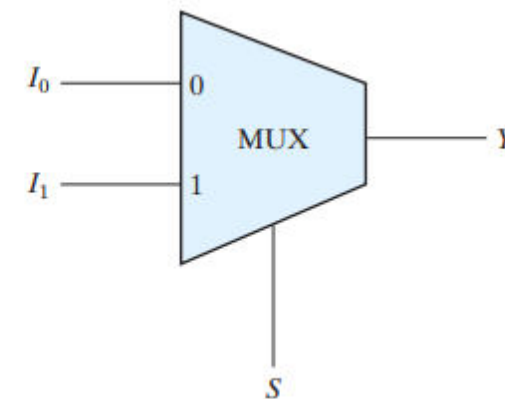


Multiplexer

- Also called as ***data selector***
- A combinational circuit:
 - Selects from one of many input lines and directs it to a single output line
 - Selection is controlled by a set of selection lines
 - There are 2^n input lines and n selection lines
- Often labeled as ***MUX***

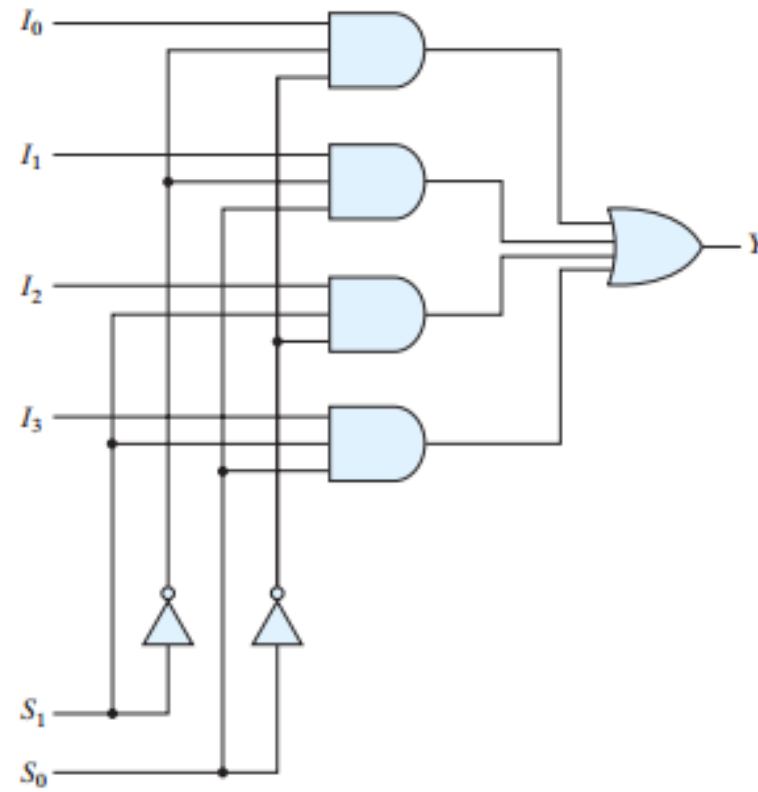


(a) Logic diagram



(b) Block diagram

4-to-1 MUX



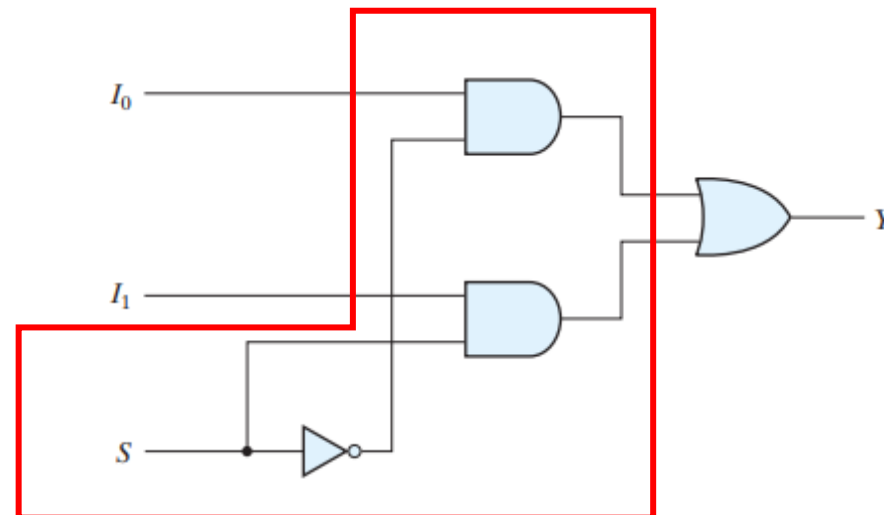
(a) Logic diagram

S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

(b) Function table

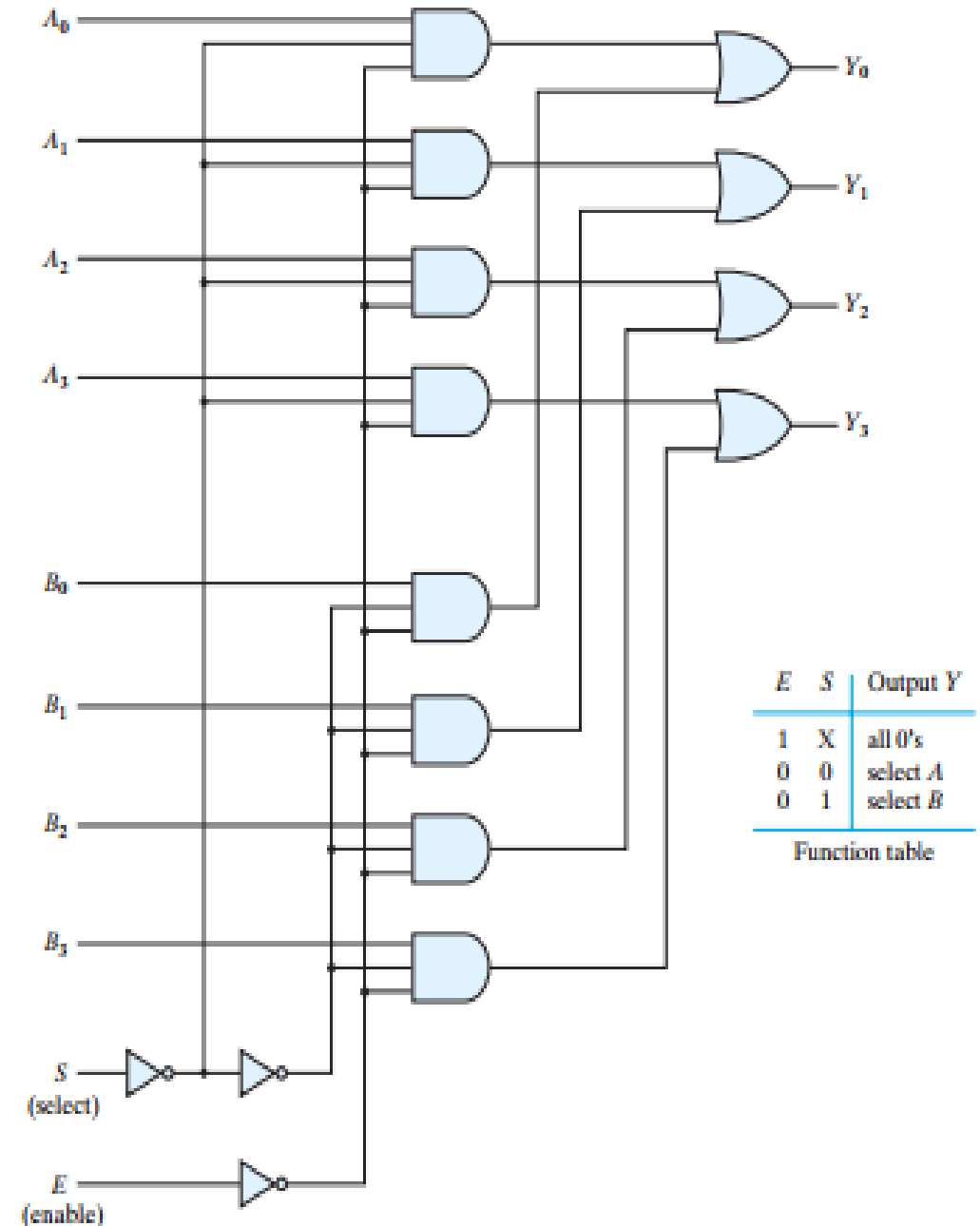
Multiplexer from Decoder

- The **AND gates** and **inverters** in the multiplexer resemble a decoder circuit
 - Indeed, they decode the selection input lines
 - In general, a **2^n -to-1-line multiplexer** is constructed from an **n -to- 2^n decoder** by adding **2^n input lines** to it, one to each AND gate.
 - The outputs of the AND gates are applied to a single OR gate.



Circuit Implementation with Multiplexer

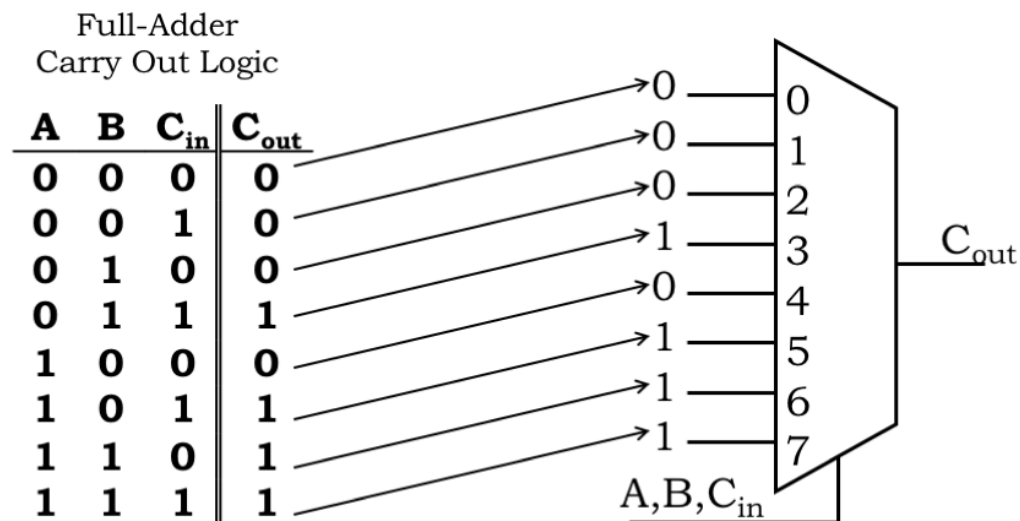
Multiplexer circuits (**more than one**) can be combined with common selection inputs to provide **multiple-bit selection logic** for all of them.



Circuit Implementation with Multiplexer

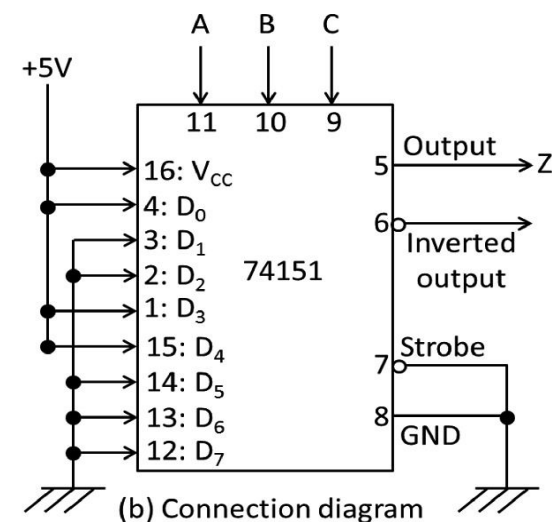
Normal way

- The individual minterms can be selected by the data inputs of the multiplexer
 - Implement a Boolean function of n variables with a multiplexer that has n selection inputs and 2^n data inputs – one for each minterm.



Select inputs				Data inputs
A	B	C	Z	D _i
0	0	0	1	D ₀
0	0	1	0	D ₁
0	1	0	0	D ₂
0	1	1	1	D ₃
1	0	0	1	D ₄
1	0	1	0	D ₅
1	1	0	0	D ₆
1	1	1	0	D ₇

(a) Truth table



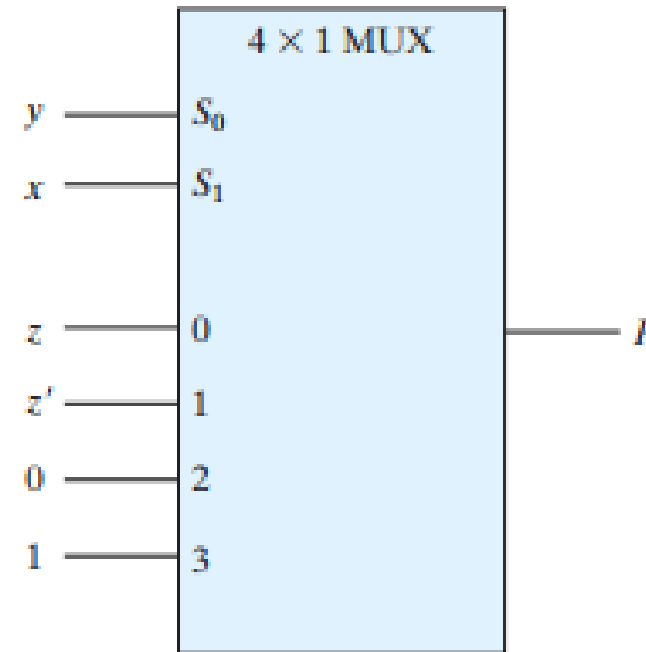
Circuit Implementation with Multiplexer

More efficient way (Example 1)

- $F(x, y, z) = \sum(1, 2, 6, 7)$

x	y	z	F	
0	0	0	0	$F = z$
0	0	1	1	
0	1	0	1	$F = z'$
0	1	1	0	
1	0	0	0	$F = 0$
1	0	1	0	
1	1	0	1	$F = 1$
1	1	1	1	

(a) Truth table



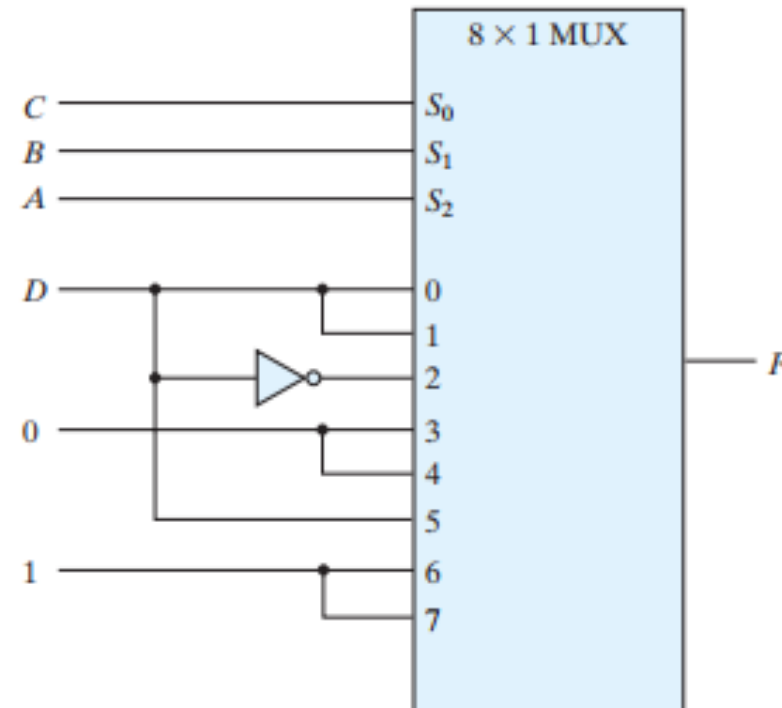
(b) Multiplexer implementation

Circuit Implementation with Multiplexer

More efficient way (Example 2)

- $F(w, x, y, z) = \sum(1, 3, 4, 11, 12, 13, 14, 15)$

A	B	C	D	F	
0	0	0	0	0	$F = D$
0	0	0	1	1	
0	0	1	0	0	$F = D$
0	0	1	1	1	
0	1	0	0	1	$F = D'$
0	1	0	1	0	
0	1	1	0	0	$F = 0$
0	1	1	1	0	
1	0	0	0	0	$F = 0$
1	0	0	1	0	
1	0	1	0	0	$F = D$
1	0	1	1	1	
1	1	0	0	1	$F = 1$
1	1	0	1	1	
1	1	1	0	1	$F = 1$
1	1	1	1	1	

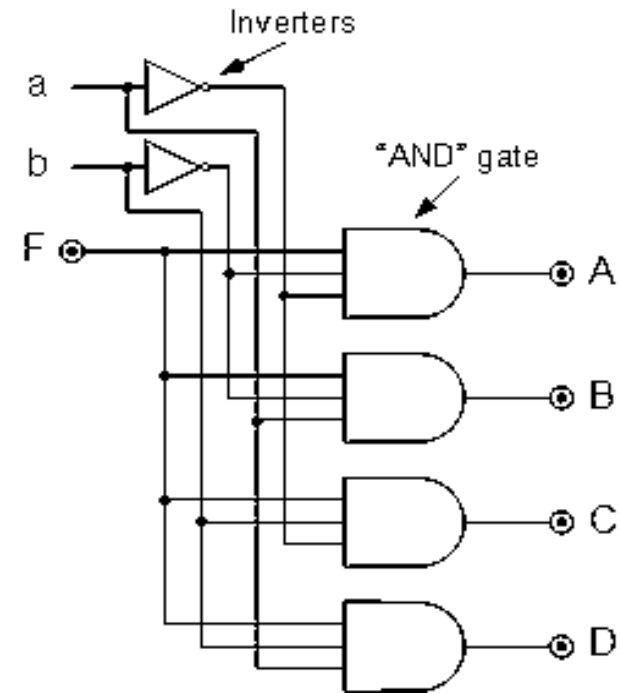
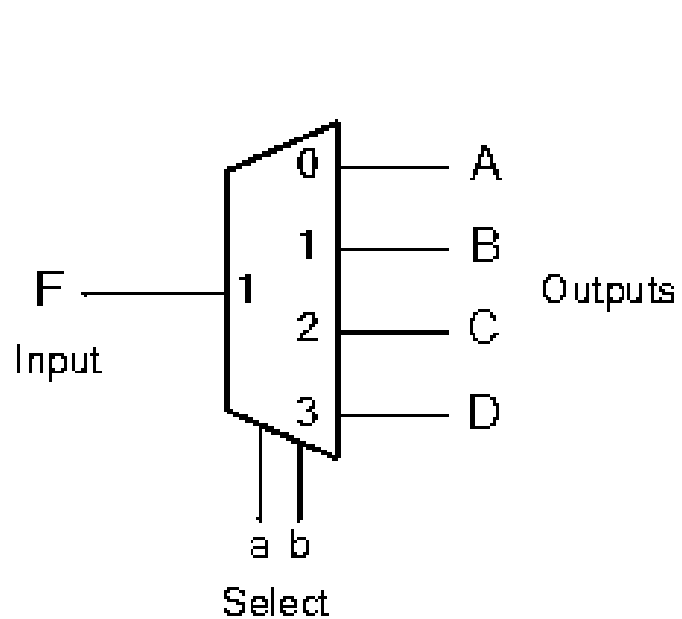




Demultiplexer

- Also called as ***data distributor***
- **Definition:** A circuit that receives information from a single line and directs it to one of 2^n possible output lines
- Selection of a specific output will be controlled by **bit combination of n selection lines**
- A **decoder** could be used as a demultiplexer
 - Here, the **enable input** in decoder works as the original input in demultiplexer
 - Connected with each of the outputs , but only one will be activated specified by the binary combination of the input bits of the decoder
 - Decoder with enable input referred as a ***decoder-demultiplexer***

Demultiplexer: Symbol and Circuit



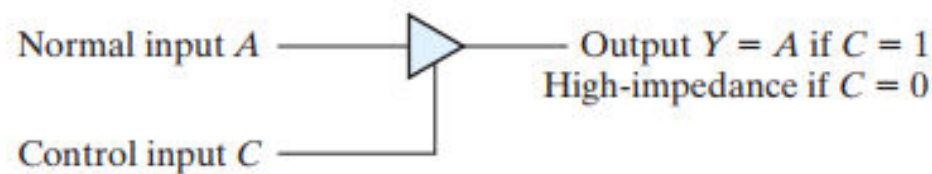
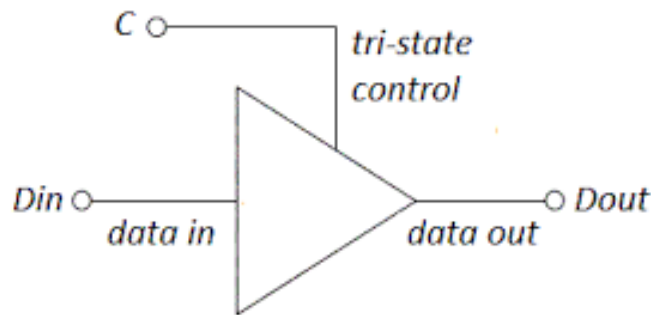


Three State Gates

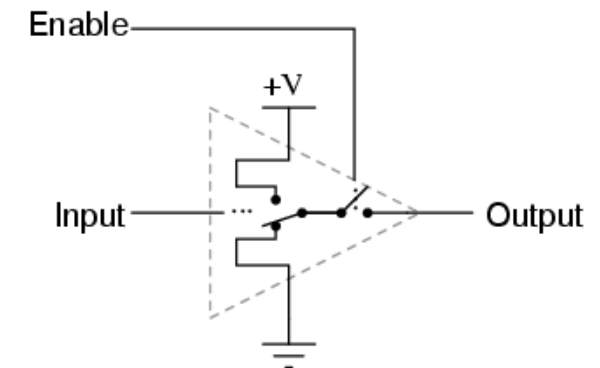
- In digital electronics, **three-state logic**, **tri-state logic**, or **3-state logic** allows an **output** or **input** pin to be in a **high impedance state (additional)**, which means effectively removing that output or input from the circuit.
 - This is in addition to the **0** and **1** logic levels that are used in digital electronics.
 - Sometimes denoted as **Hi-Z**
- Tri-state gates have additional circuitry via which the gate outputs can be **enabled or disabled**.
- This allows **multiple circuits** to share the **same output** line or lines
 - Such as a communication **bus** which cannot listen to more than one device at a time but they are connected all together with tri-states.
- Sometimes, they are implemented with **conventional logic** like AND or NAND
 - But most commonly used with buffer gate

Tri-state Buffer - Symbol

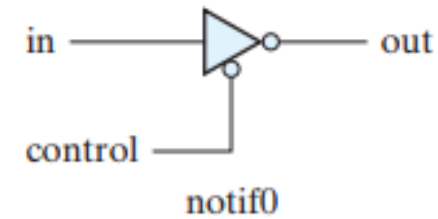
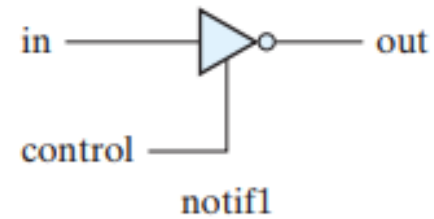
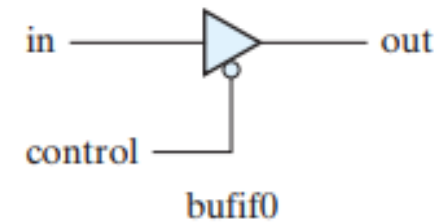
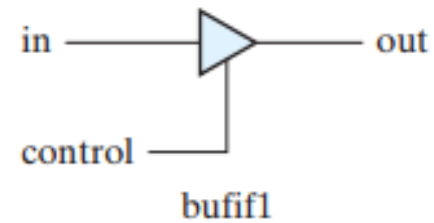
- A normal buffer with **additional control line** entering the bottom/top of the symbol
 - This tri-state buffer has a normal input, an output and a control input which determines the state of the output
 - When the control input is 1, the output is enabled and the buffer gate behaves like a conventional buffer.
 - When the control input is 0, the output is disabled and the gate goes to a high-impedance (Hi-Z) state.
 - Because of this feature, a large number of tri-state gate outputs are connected with wires to form a common line **without endangering the loading effects**



Tristate buffer gate



Tri-state gate – More gates

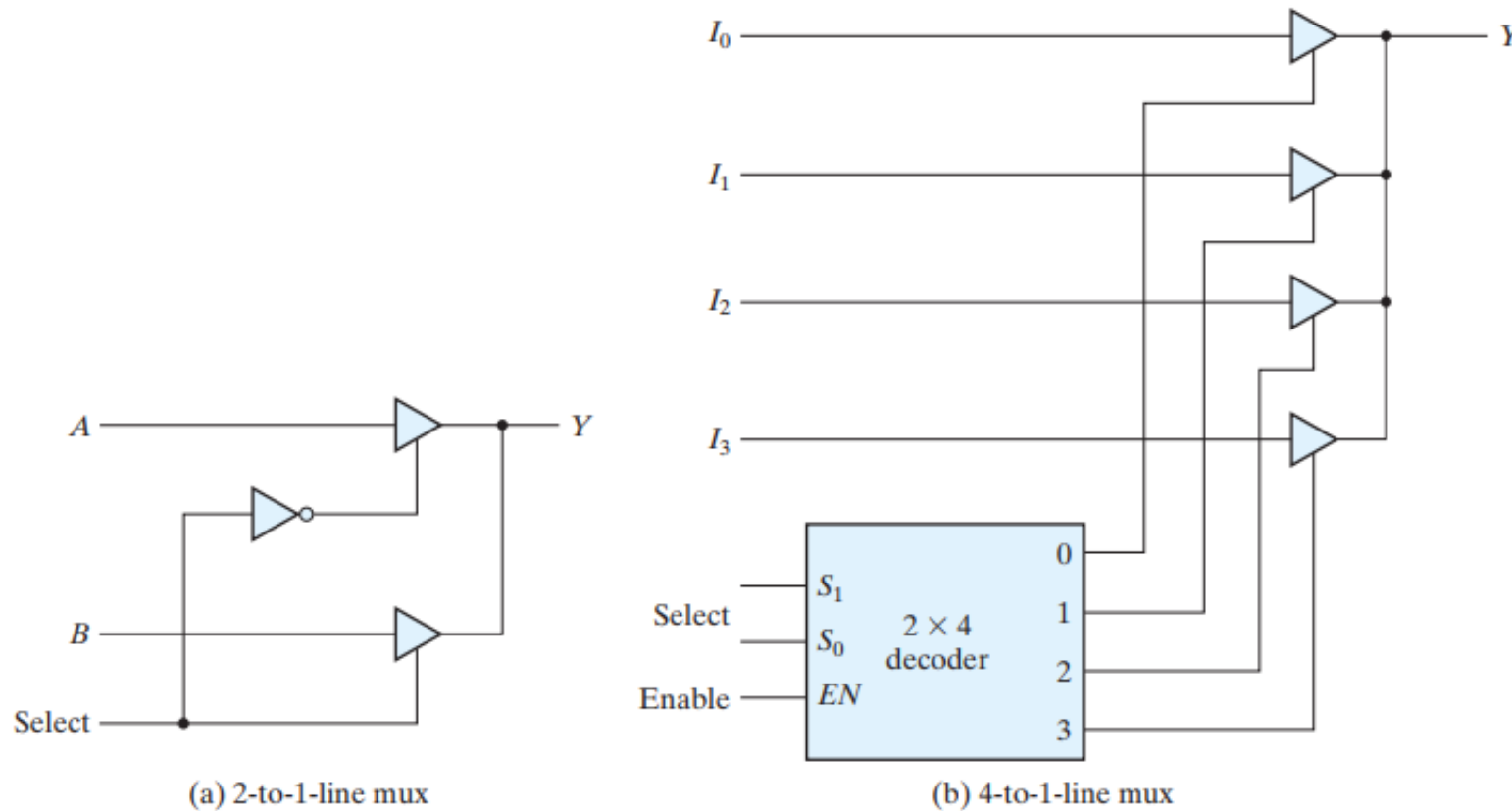




Consequence of a High Impedance State (Hi-Z)

- The third state is a high-impedance state (Hi-Z) **in which**
 - The logic behaves like an open circuit
 - Which means that the output appears to be disconnected
 - The circuit has no logic significance
 - The circuit connected to the output of the three-state gate is not affected by the inputs to the tri-state gate.

Multiplexer with **Tri-state Buffer**



Note: Outputs are connected together to form a single output line.
Also, if **enable input** of the tri-state is 0, all the outputs are 0 and bus line is in a **Hi-Z** state

abc



abc

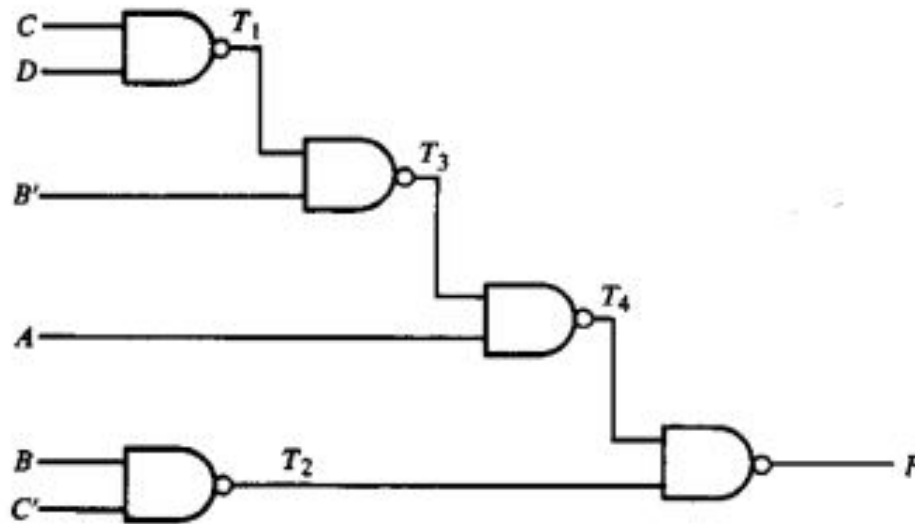


NAND Implementation: Analysis Procedure

- **Reverse process** – **starts** from given NAND logic diagram and **ends** with a Boolean expression or truth table
- Specialty – Application of De Morgan's Law
- **Derivation of Boolean Function from NAND Logic Diagram**
 - All gates' outputs are **labeled with arbitrary** symbols
 - The **Boolean functions for the outputs** of gates that receive only external inputs are derived. It may follow **De Morgan's law** to make it convenient to use
 - **Boolean output functions of gates** which have inputs from previously derived functions are determined in consecutive order **until the output is expressed in terms of input variables**

NAND Imp.: Analysis Procedure ...

Example:



$$T_1 = (CD)' = C' + D'$$

$$T_2 = (BC')' = B' + C$$

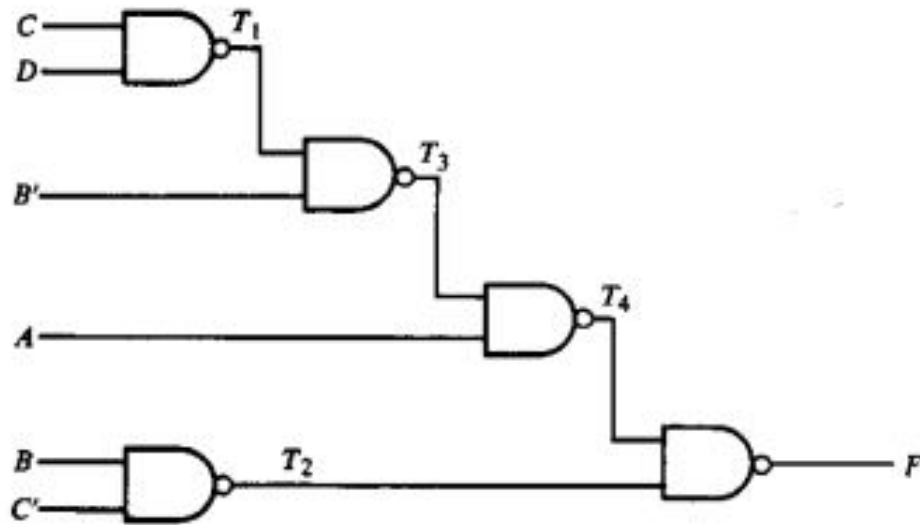
$$\begin{aligned} T_3 &= (B'T_1)' = (B'C' + B'D')' \\ &= (B + C)(B + D) = B + CD \end{aligned}$$

$$T_4 = (AT_3)' = [A(B + CD)]'$$

$$\begin{aligned} F &= (T_2T_4)' = \{(BC')'[A(B + CD)]'\}' \\ &= BC' + A(B + CD) \end{aligned}$$

NAND Imp.: Derivation of Truth Table

Same as before.



Truth Table for the Circuit of Figure 4-14

A	B	C	D	T ₁	T ₂	T ₃	T ₄	F
0	0	0	0	1	1	0	1	0
0	0	0	1	1	1	0	1	0
0	0	1	0	1	1	0	1	0
0	0	1	1	0	1	1	1	0
0	1	0	0	1	0	1	1	1
0	1	0	1	1	0	1	1	1
0	1	1	0	1	1	1	1	0
0	1	1	1	0	1	1	1	0
1	0	0	0	1	1	0	1	0
1	0	0	1	1	1	0	1	0
1	0	1	0	1	1	0	1	0
1	0	1	1	0	1	1	0	1
1	1	0	0	1	0	1	0	1
1	1	0	1	1	0	1	0	1
1	1	1	0	1	1	1	0	1
1	1	1	1	0	1	1	0	1

NAND Imp.: Derivation of Truth Table ...

Now we can use K-map to get the simplified expression of the given NAND logic circuit.

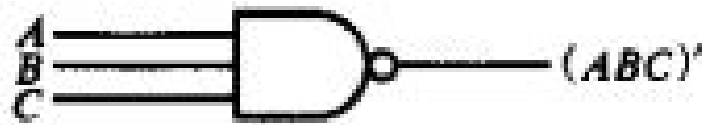
		CD		C	
		00	01	11	10
AB	00				
	01	1	1		
	11	1	1	1	1
	10			1	

$\underbrace{\hspace{10em}}_D$

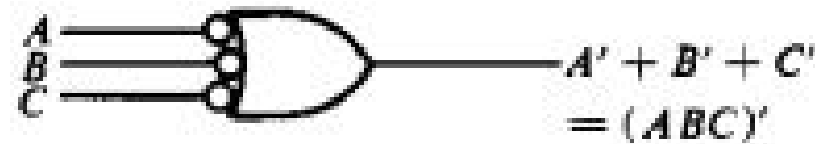
$$F = AB + ACD + BC' = A(B + CD) + BC'$$

NAND Imp.: Block Diagram Transformation

- Without employing **De Morgan's Law**.
- We use two **alternate graphic symbols** of NAND gates



(a) AND-invert

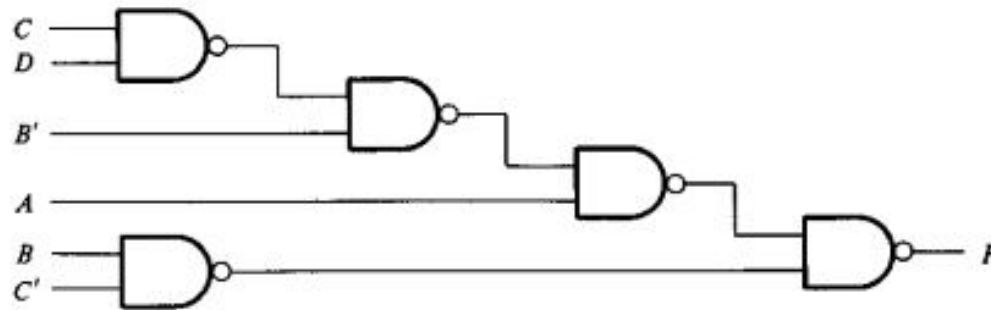


(b) invert-OR

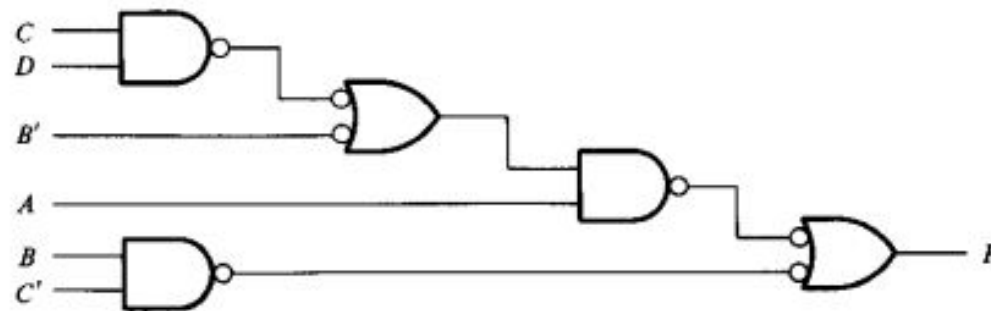
NAND Imp.: Block Diagram Transformation...

- **NAND logic diagram – AND-OR diagram conversion**
 - A change in symbols from AND-invert to an invert-OR in **alternate levels** of gates.
 - **At first** we start from the **last level**, to be changed to an **invert-OR symbol**.
 - These changes produce **pairs of circles along the same line** – can be removed as they can be cancelled.
 - **One input AND or OR gate** can be changed to an inverter.
 - **Circle with external input** can be changed to corresponding complemented variable.

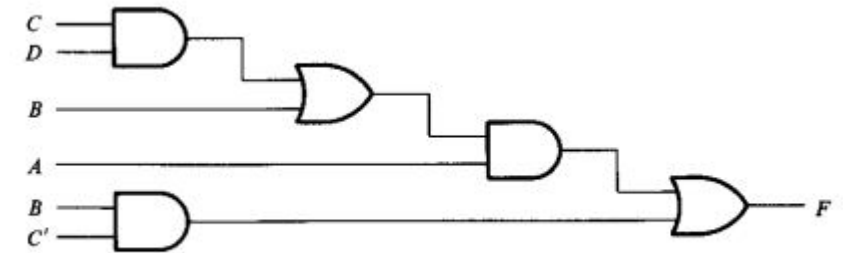
NAND Imp.: Block Diagram Transformation...



(a) NAND logic diagram

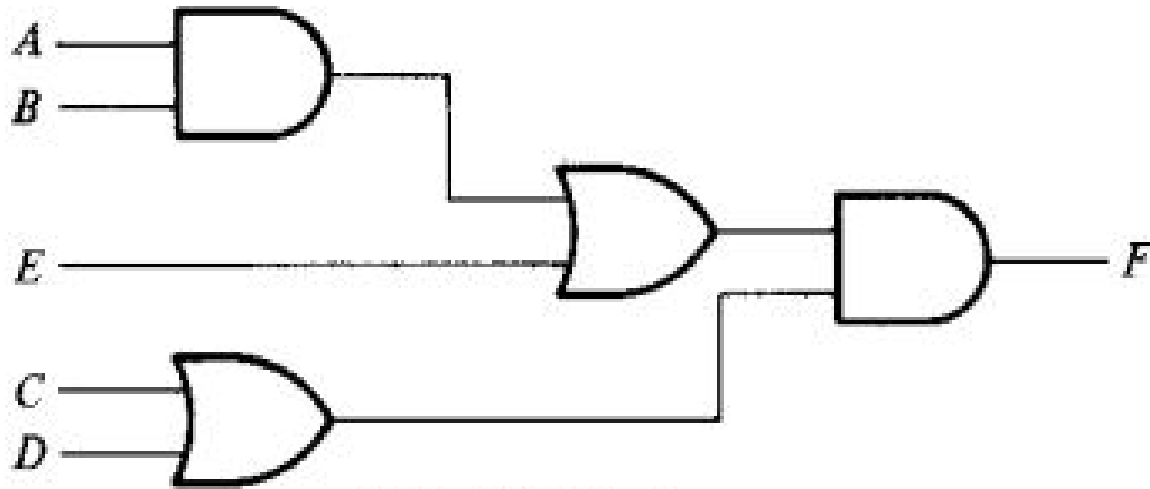


(b) Substitution of invert-OR symbols in alternate levels

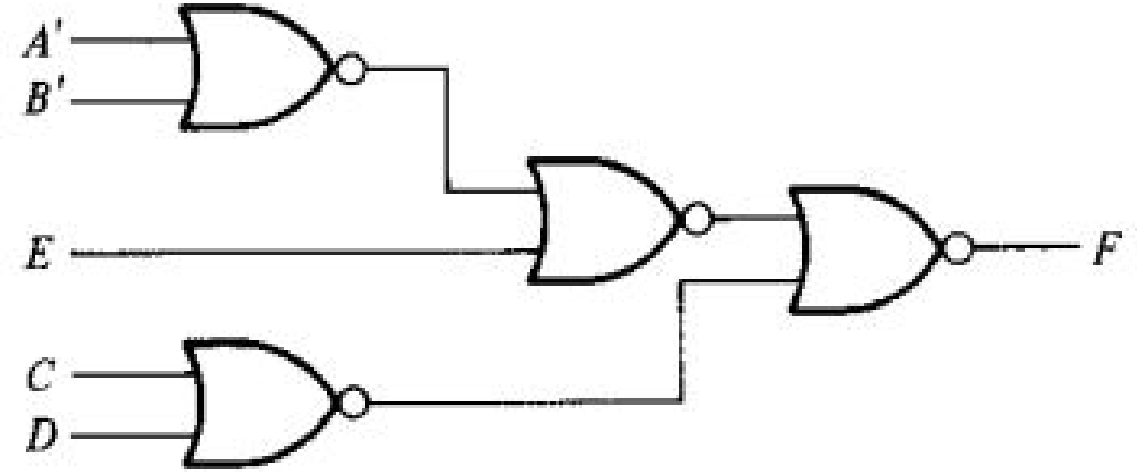


(c) AND-OR logic diagram

Block Diagram Method: NOR Imp.



(a) AND-OR diagram



(c) Alternate NOR diagram

In general, the number of NOR gates required to implement a function is equal to the number of AND-OR gates, except for an occasional inverter. It will be applicable when both normal and complement inputs are available.



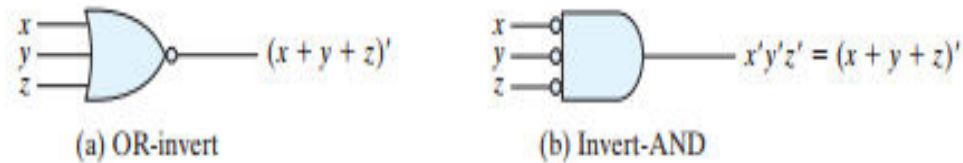
NOR Implementation: Analysis Procedure

- To derive the Boolean function from a logic diagram, we mark the outputs of various gates with arbitrary symbols.
- By repetitive substitutions, we obtain the output variables as a function of the input variables.

To obtain the truth table from a logic diagram without deriving the Boolean function first, we form a table listing the n input variables with 2^n rows of 1s and 0s. The truth table of various NOR gate outputs is derived in succession until the output truth table is obtained.

Block Diagram Transformation

To convert a NOR logic diagram to its equivalent AND-OR logic diagram, we use the two symbols for NOR gates.

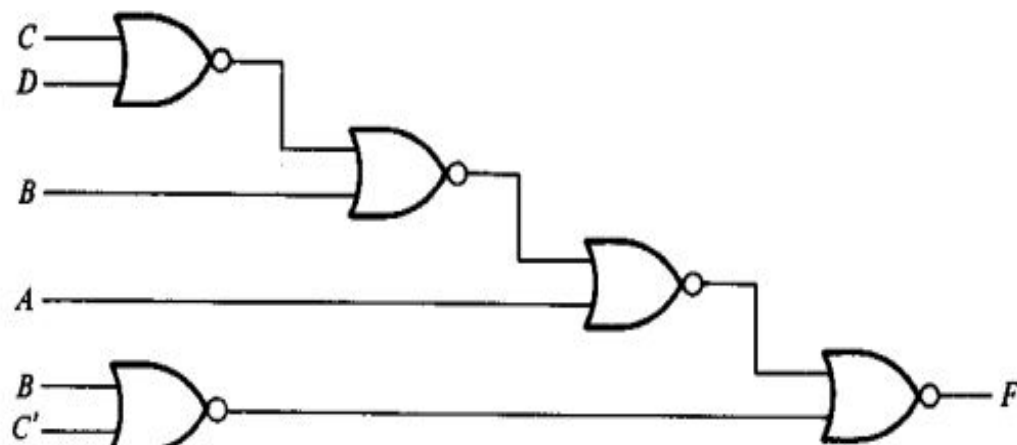


The conversion of a NOR logic diagram to an AND-OR diagram can be achieved through a change in symbols from OR-invert to invert-AND starting from the last level and in alternative levels.

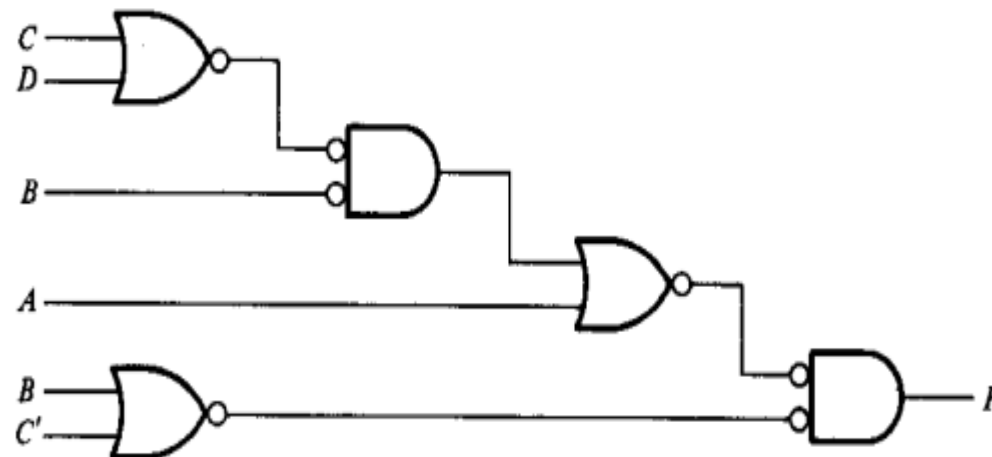
Pairs of small circles along the same line must be removed.

A one-input AND or OR gate should be removed but if it has a small circle at the input or output, it should be converted to an inverter.

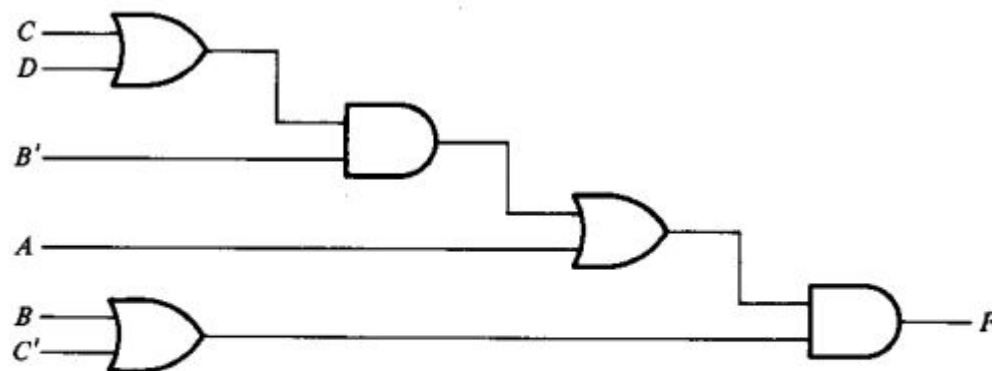
Block Diagram Transformation



(a) NOR logic diagram



(b) Sustituting invert-AND in alternate levels



(c) AND-OR logic diagram

FIGURE 4-20

Conversion of NOR diagram to AND-OR